

基于滚动预测与储能协同调度的微网购电策略优化研究

摘 要

本文针对微网与外部电网的电力调控问题，建立能量平衡与储能状态约束下的购电优化模型，研究确定性日计划、负载与光伏变化、预报更新和波动电价四种条件下的调度策略。通过线性规划、滚动预测和历史残差情景，协调常规购电、储能充放电与紧急补购。

问题一中，建立以日购电费用最小为目标的线性规划模型，在首尾储电量相等的条件下，求得日购电量 59,482.70 kWh，最低费用 35,126.95 元，较无储能方案节省 26.90%。通过原始与对偶目标的一致性检验验证该结果。

问题二中，采用历史数据训练岭回归预测器，以三条残差情景制定日前计划，并根据当前净负荷执行储能调节。2 月至 12 月总费用为 1,450.61 万元。问题三中，建立考虑增购、取消和紧急购电的滚动调整模型；采用 0、6、12、18 时全部预报时，总费用为 1,396.89 万元，较仅用 0 时预报降低 4.61%，紧急购电量降低 87.18%。

问题四中，将分时固定电价推广为波动电价。在当天价格曲线已知的条件下，日前计划与滚动调整方案的费用分别为 1,524.94 万元和 1,460.88 万元；仅用历史价格预测时，对应费用为 1,531.96 万元和 1,473.23 万元。两类结果分别反映不同的信息条件。

结果表明，储能移峰与预报更新在所测策略中能够降低购电支出。模型通过能量守恒、容量、功率和费用核算检验；除问题一外，结果为给定策略的历史回放费用，其适用范围受预测误差、结算约定与实时控制规则限制。

关键词：微网；储能调度；线性规划；滚动预测；情景优化

第 1 章 问题重述

1.1 问题背景

微网由小区负载、光伏发电设备和储能设备组成，并通过外部电网补充电能。光伏出力与用电需求在时间上的差异，使得单纯依靠即时购电难以充分利用低价电能和本地发电。储能设备能够在低价或光伏富余时充电，在高价或供能不足时放电，从而改变外网购电的时间分布。

本研究考虑的调控目标是在持续满足小区负载的前提下，尽可能降低购电费用。系统每 10 分钟记录一次功率，储能最大容量为 12000 kWh，允许储电量为 1200 至 10800 kWh，最大充放电功率为 5000 kW，初始储电量为 6000 kWh。预测误差与电价变化进一步影响日前计划和后续调整，需要在统一的能量约束下进行分析。

1.2 问题要求

问题 1：每天电价和负载相同，已知一天的光伏预测，在 0 时制定日购电计划，并满足日初与日末储电量相等。依据附件 1 给出指定购电时段、六个储能统计区间及全天费用。

问题 2：负载和光伏功率随时间变化，每天 0 时制定计划，其余常规费用按计划结算。供能不足部分按当时电价的 5 倍紧急购电，需利用附件 2 给出全年计划及指定日期结果。

问题 3：每天 0、6、12、18 时获得未来 24 小时的整点光伏预报，允许调整未执行计划，并计入增购或取消交易的相关费用。利用附件 3 建立滚动调整模型，分析其他预报时刻的应用价值。

问题 4：利用附件 4 的波动电价重新计算问题 2 和问题 3，分别给出日前计划和滚动调整结果，并说明电价信息在决策时是否可用。

第 2 章 问题分析

2.1 问题 1 的分析

问题 1 的核心在于将电能的跨时段转移转化为带储能状态的费用最小化问题。由于负载、电价和光伏预测均已给定，购电量与充放电量可以作为连续决策变量。功率约束限制单个时段可转移的电量，容量约束限制跨时段积累的电量，首尾状态相等则防止通过消耗初始库存获得虚假的低费用。

在变量定义一致的条件下，供需平衡和储能动态均可写为线性关系。因此，采用线性规划直接求解，并以独立模型的对偶目标验证最优性。同时结合日内曲线解释低价充电、高价放电及部分时段仍需购电的原因。

2.2 问题 2 的分析

问题 2 的困难在于日前决策不能使用尚未发生的实际负载和光伏数据。计划过低会增加高价紧急购电，计划过高又可能造成已付费电量弃置；储能则在二者之间提供缓冲。因此，预测误差应进入购电决策，而不宜仅把一条点预测当作确定性输入。

本文先比较不同历史预测方法，再利用完整日残差构造净负荷情景，以共同的前购电量连接各情景。运行时根据当前观测执行充放电，并将日末状态传递至下一日。这样既能保持决策的时间顺序，又能通过实际账单评价预测与调度的综合效果。

2.3 问题 3 的分析

问题 3 在日前计划基础上增加了预报更新和交易调整。模型需要解决两个衔接问题：一是将整点预报映射为 10 分钟序列，二是用更新时刻的实际储电量连接剩余时段的优化。已执行的交易不可回改，尚未执行的计划则需权衡预报改善带来的收益与调整费用。

由于一次调整会影响后续储能状态，各发布时间的作用不能简单相加。本文采用逐步加入 6、12、18 时更新的嵌套策略进行比较，分析其总费用和紧急购电变化。对于题面未提供预报的其他时刻，只讨论引入条件，不虚构其预测效果。

2.4 问题 4 的分析

波动电价使购电时间的选择同时依赖净负荷和价格判断。附件 4 给出的是实际发生的价格，不能仅因数据可见就认定当日零点已知全天曲线。因此，应将已知当天价格的条件基准与仅利用历史价格的预测方案分开求解，并统一按实际交易价格结算。

本文沿用问题 2 和问题 3 的物理约束与执行规则，只改变决策使用的价格输入。通过并列比较两种信息条件，区分价格预测误差与预报更新的影响，使结果解释与模型假设一致。参见图 1。

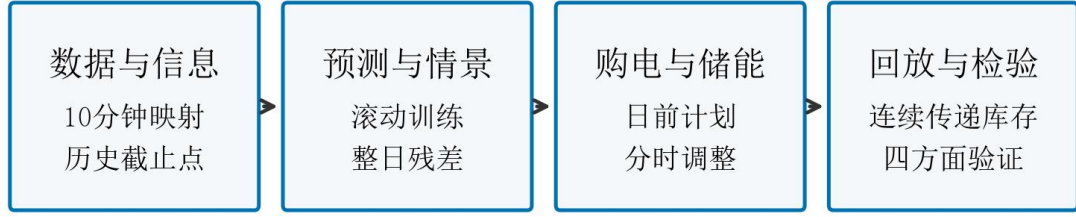


图 1 四个问题的建模与验证流程

第 3 章 符号说明

主要符号及单位

符号	含义	单位
d, t	日期与日内时段索引	—
Δt	时段长度，取 1/6 小时	h
$L_{d,t}, P_{d,t}^{pv}$	小区负载与光伏发电功率	kW
$E_{d,t}^L, E_{d,t}^{pv}$	负载用电量与光伏发电量	kWh
$p_{d,t}, \hat{p}_{d,t}$	实际电价与预测电价	元/kWh
$G_{d,t}^0, G_{d,t}^k$	日前计划量与第 k 次调整后计划量	kWh
$C_{d,t}, D_{d,t}$	母线侧充电量与放电量	kWh
$S_{d,t}$	时段起点的储电量	kWh
$R_{d,t}, W_{d,t}$	紧急购电量与弃置电量	kWh
η_c, η_d	充电效率与放电效率	—
$N_{d,t}, N_{d,t}^{(s)}$	实际净负荷电量与情景净负荷电量	kWh
$\delta_{d,t}^{k,+}, \delta_{d,t}^{k,-}$	第 k 次调整的增购量与减购量	kWh
J, κ	购电总费用与终端偏差系数	元；元/kWh

第 4 章 模型假设

1. 时间离散假设：每个功率采样值代表此前 10 分钟区间的平均功率，时间标签为右端点。本文据此转换功率与电量，并在全文保持统一映射。

2. 储能效率假设：充电和放电效率分别取 90%，往返效率为 81%。充电量和放电量均在母线侧计量；若 90%指往返效率，则另作敏感性分析。

3. 设备与交易假设：忽略储能自放电、老化和额外运维费用，不设置售电收益及外网购电功率上限。允许弃光和已付费富余电量弃置。

4. 状态连续假设：问题 1 首尾储电量均为 6000 kWh；问题 2 至问题 4 从 1 月 1 日的 6000 kWh 连续运行，不对每天施加首尾相等条件。

5. 信息可用性假设：未来实际负载和光伏数据仅用于事后评价。预测使用此前数据，实时执行使用当前观测；问题 4 分别研究已知当日电价和仅用历史价格的条件。

6. 调整结算假设：取消电量退回原款，并支付 50% 的违约费用；每次增减按上一有效计划计算。题面未明确的退款和多次调整基准通过该约定补充，并检验不退款情形。

7. 初始化假设：1 月 1 日无历史样本时，以附件 1 负载、光伏与电价曲线冷启动；1 月用于预运行和参数选择，正式统计区间为 2 月 1 日至 12 月 31 日。

第 5 章 模型建立与求解

5.1 数据处理与统一物理约束

5.1.1 数据整理与时间映射

附件 1 提供日内电价、负载和光伏预测；附件 2 提供全年实际负载与光伏；附件 3 提供分时发布的未来整点预报；附件 4 提供全年实际电价。各序列统一按日期和日内时段建立索引。数值区域共 193152 个数据，未发现缺失、负值或非有限数；附件 3 组内空白日期采用该组首行日期，不作为预报缺失处理。参见图 2。

$$\Delta t = \frac{1}{6}, E_{d,t}^L = L_{d,t} \Delta t, E_{d,t}^{pv} = P_{d,t}^{pv} \Delta t \quad (1)$$

例如 10:10 采样行对应 10:00—10:10 区间。结果模板末行或末列的次日标记存在歧义，本文按程序实际映射将其标为本日 00:00—00:10，并保留其他列顺序。这一时间解释属于建模约定，不视为官方勘误。四小时储能统计由连续 24 个 10 分钟时段求和得到。

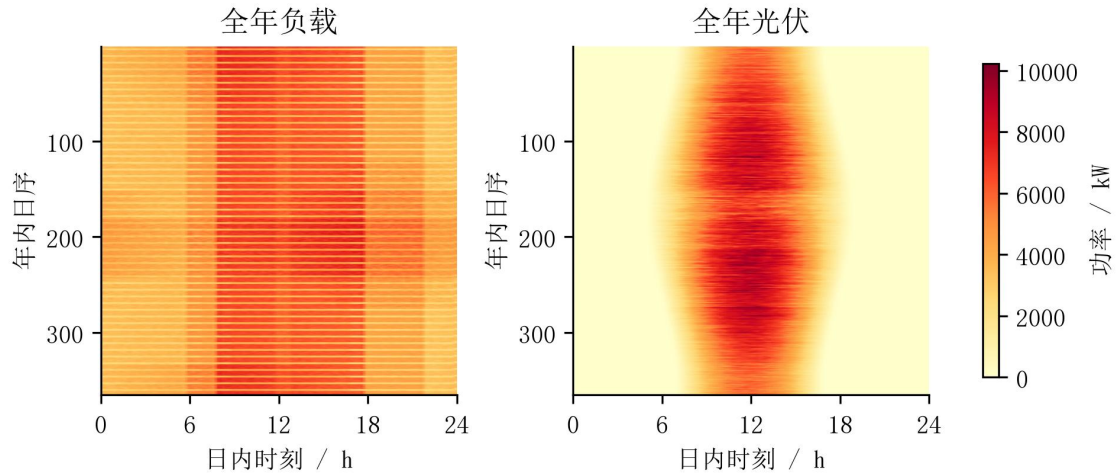


图 2 负载与光伏的全年日内分布

5.1.2 能量平衡模型

微网母线的供给包括常规购电、光伏发电、储能放电和紧急购电；其去向为负载、储能充电与弃置。引入非负弃置变量后，题目中供电不低于负载的要求可以统一表达为：

$$G_{d,t} + E_{d,t}^{pv} + D_{d,t} + R_{d,t} = E_{d,t}^L + C_{d,t} + W_{d,t} \quad (2)$$

弃置电量不产生收入，已经购入的富余电量仍须支付费用。因此，不能用负购电量代替弃置，也不能将全部富余电量当作可售电量。

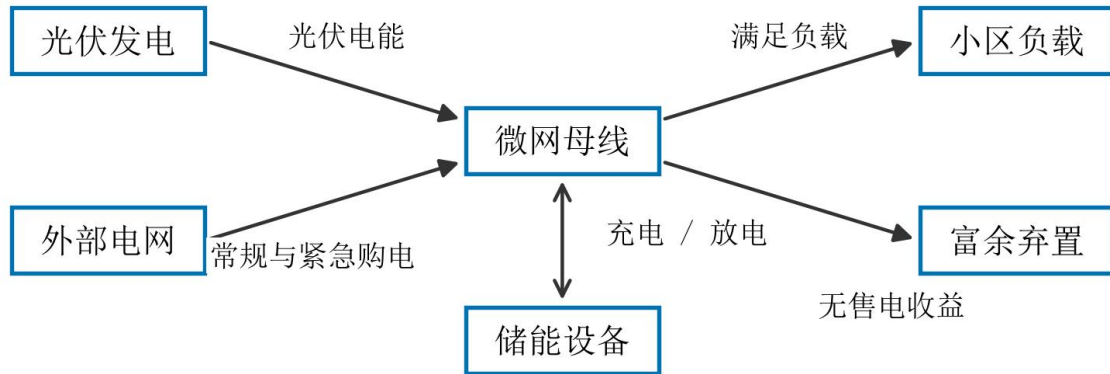


图 3 微网能量流及储能双向调节

能量流图将母线的供给与去向对应到平衡方程。储能同时具有吸收和释电能两种运行方向，实际时段按状态选择其一；富余弃置不产生售电收入。参见图 3。

5.1.3 储能状态与运行边界

充电时，输入电量的一部分转化为储能库存；放电时，为向母线提供规定电量，库存减少量需要考虑效率损失。状态转移与运行边界分别为：

$$S_{d,t+1} = S_{d,t} + \eta_c C_{d,t} - \frac{D_{d,t}}{\eta_d} \quad (3)$$

$$1200 \leq S_{d,t} \leq 10800, 0 \leq C_{d,t}, D_{d,t} \leq \frac{5000}{6} \quad (4)$$

$$G_{d,t}, R_{d,t}, W_{d,t} \geq 0, S_{d+1,0} = S_{d,144} \quad (5)$$

后续各问均在上述约束下求解。功率上限必须逐时段检查，不能仅凭四小时汇总电量推断设备是否越限；连续运行还需检查相邻日期的首尾状态是否一致。相关关系见式(2)、(3)、(4)。

5.2 问题 1 的模型建立与求解

问题一要求在已知日内曲线的条件下确定购电与储能安排。本文先建立日购电费用目标，再施加首尾储能一致约束，通过线性规划得到完整的 10 分钟调度，最后按题设格式汇总。

5.2.1 目标函数与边界条件

$$\min J_1 = \sum_{t=0}^{143} p_t G_t, R_t = 0, S_0 = S_{144} = 6000 \quad (6)$$

决策变量由 144 个购电量、144 个充电量、144 个放电量、144 个弃置量和 145 个储电状态组成。结合统一物理约束，模型为线性规划，可采用 HiGHS 求解器计算[1,2]。为稳定选择解，调度程序加入极小的吞吐惩罚；独立最优性检验则直接最小化真实购电费。相关关系见式(6)。

$$J_\varepsilon = J_1 + 10^{-7} \sum_{t=0}^{143} (C_t + D_t) \quad (7)$$

在正电价且允许弃置的条件下，若某时段同时充放电，将充电减少 a 、放电减少 $\eta_c \eta_d a$ ，储能变化保持不变，母线富余增加 $(1 - \eta_c \eta_d)a$ 。通过减少购电或增加弃置可恢复平衡，真实费用不增加而吞吐惩罚下降。因此，存在不同时充放电的最优解，无需在本模型中额外引入整数变量。

5.2.2 求解步骤

Step1: 数据与参数初始化

将附件 1 功率换算为时段电量，设置效率、容量、功率及 6000 kWh 首尾状态，按统一时间映射排列 144 个区间。

Step2: 约束矩阵构造与求解

逐时段建立能量平衡和储能转移等式，设置各变量上下界，求解线性规划并提取购电、充放电、弃置与储电轨迹。

Step3: 独立验证与结果汇总

另建仅含真实费用目标的线性规划，比较原始与对偶目标，并用双单纯形和内点方法交叉检查；通过后汇总指定区间和全天指标。

5.2.3 求解结果及分析

表 1 问题 1 的优化结果

计算项目	数值结果
全天购电量 / kWh	59,482.6990
全天购电费 / 元	35,126.948589
无储能费用 / 元	48,052.046591
费用降低比例	26.90%
原始与对偶目标差 / 元	7.28×10^{-12}
全天弃置电量 / kWh	0.0000

表 2 问题 1 指定时段购电量及全天结果

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.0000	12:00-12:10	480.4124	14:00-14:10	0.0000
16:00-16:10	445.4317	18:00-18:10	531.8940	20:00-20:10	0.0000
全天购电量	59,482.6990	全天购电费	35,126.95		

表 3 问题 1 储能充放电量与首尾储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00-4:00	4,500.0000	0.0000	4:00-8:00	833.3333	6,365.8412
8:00-12:00	4,787.9643	1,702.9970	12:00-16:00	5,286.0352	91.1014
16:00-20:00	0.0000	5,780.1319	20:00-24:00	5,333.3333	2,859.8681
0:00 储电量	6000.0000		24:00 储电量	6000.0000	

表中电量单位为 kWh、费用单位为元。全天充电量为 20,740.6661 kWh，放电量 16,799.9396 kWh，首尾储电量均为 6000 kWh。最大充电功率达到 5000 kW，最大放电功率为 4,293.9182 kW，均满足约束。

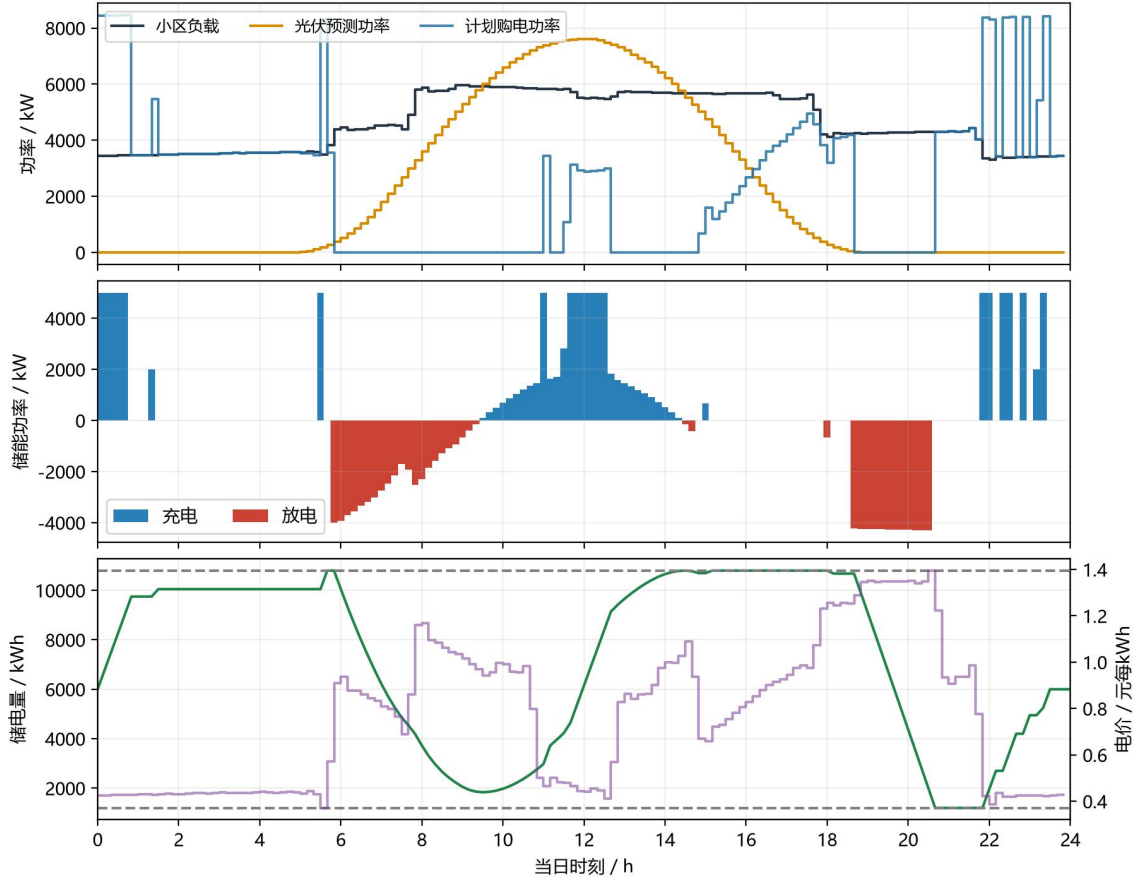


图 4 问题 1 购电与储能调度结果

从计算结果可以看出，储能通过低价充电和高价放电降低了外网购电支出。中午即使光伏接近或超过负载，较低电价仍可能使购电充电具有经济性，因此 12:00—12:10 的购电量不必为零。晚间高价时段的放电受功率和库存限制，部分需求仍须由外网补充。参见图 4。

在首尾储能相等时，全天供给与需求之差应等于充放电损耗，其检验关系为：

$$\sum_t (G_t + E_t^{pv} - E_t^L - W_t) = (1 - \eta_c) \sum_t C_t + (\eta_d^{-1} - 1) \sum_t D_t \quad (8)$$

独立计算的全天能量残差小于 10^{-9} kWh，两种求解算法给出相同费用。该检验说明充电量大于放电量源于效率损失，而非能量平衡错误。

5.3 问题 2 的模型建立与求解

5.3.1 基于历史数据的功率预测

未来负载与光伏未知时，预测必须先于调度。本文比较历史同时间均值、7 日、14 日、30 日移动平均、指数加权移动平均和岭回归六种方法，并采用滚动起点验证[3]。岭回归输入包括前 1 日、前 7 日、近 7 日均值、周周期正余弦和截距，拟合窗口最多 60 天，功率按 5000 kW 缩放。

$$\hat{\beta} = (X^T X + \Lambda)^{-1} X^T y \quad (9)$$

$$\hat{y}_{d,t} = 5000 x_{d,t}^T \hat{\beta} \quad (10)$$

非截距正则系数取 1，截距系数取百万分之一；训练不足 12 天时使用 7 日均值。预测截断为非负，其上界根据历史范围确定。1 月 17 日至 31 日用于经济选型，比较 6 种预测方法、3 种裕度和 3 种终端系数，共 54 组，选出岭回归、裕度 0 和终端系数 0.5。1 月实际预运行仍采用预先固定的参数，避免用月底选型结果改写月初状态。

表 4 不同预测方法的外样本误差

预测方法	负载 MAE	负载 RMSE	光伏 MAE	光伏 RMSE
历史均值	803.65	1,028.24	483.33	772.97
7 日均值	781.21	955.92	153.40	303.86
14 日均值	782.67	959.83	171.48	327.22
30 日均值	789.76	973.51	225.65	400.08
指数加权	838.19	1,039.77	154.73	303.83
岭回归	180.32	242.31	150.46	302.77

表中单位为 kW，评价区间为 2 月至 12 月。预测误差用于解释运行效果，未用该外样本结果反向选择参数；经济选型以 1 月验证期间的库存校正费用为依据，因而不要求误差最小的方法必然具有最低购电费用。

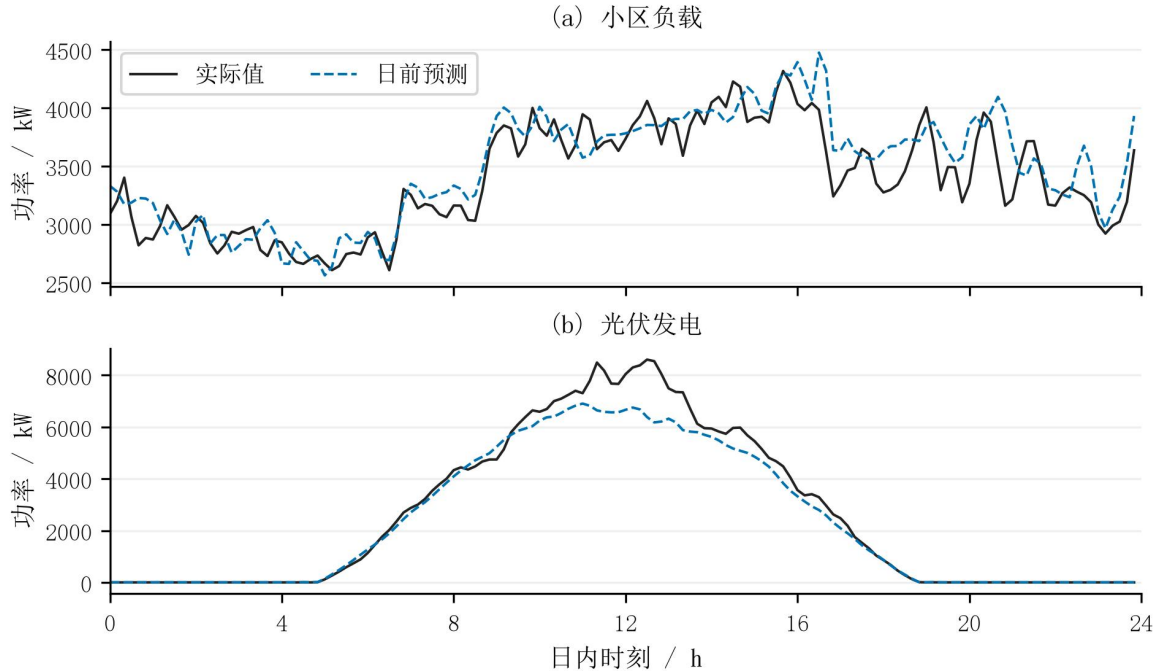


图 5 2025 年 6 月 21 日的负载与光伏预测对比

典型日曲线显示，预测能够刻画主要日内变化，但局部偏差仍然存在。负载与光伏偏差共同改变净负荷，因此购电优化还需考虑历史残差情景；单日图形仅作说明，整体精度以外样本误差表为准。参见图 5。

5.3.2 历史残差情景与日前目标函数

为描述净负荷的不确定性，保留最近 30 个完整日的预测残差，按全天残差能量排序，取约 15%、50%和 85%位置的三条整日曲线。相较于逐时独立扰动，这种构造保留了部分日内相关性，但三个等权情景并不代表已校准的真实概率分布。

$$\hat{N}_{d,t} = (\hat{L}_{d,t} - \hat{P}_{d,t}^{pv})\Delta t \quad (11)$$

$$N_{d,t}^{(s)} = \hat{N}_{d,t} + e_{d,t}^{(s)} + m\sigma_{d,t} \quad (12)$$

日前购电量在三个情景间共享，储能、紧急购电和弃置变量可随情景变化。目标函数同时考虑计划费用、情景紧急费用及终端储能偏差：相关关系见式(11)、(12)。

$$\min \left\{ \sum_t p_t G_t + \frac{1}{3} \sum_{s=1}^3 \left[\sum_t 5 p_t R_t^{(s)} + \kappa z_s \right] \right\} \quad (13)$$

$$z_s \geq S_{144}^{(s)} - 6000, z_s \geq 6000 - S_{144}^{(s)}, \kappa = 0.5\bar{p} \quad (14)$$

各情景均满足统一物理约束。这里允许情景内动作依赖整条情景，因此所得规划是用于选择日前购电的两阶段近似；它没有建立完整场景树的非预见性约束，不能视为严格的多阶段最优控制。实际电池执行另按当前观测决定。

5.3.3 实时储能控制与日际衔接

记当前实际净负荷电量为 N_t ，常规购电抵消净负荷后的余量为 x_t 。富余时优先充电，缺口时优先放电，超过储能调节能力的缺口进行紧急购电：

$$x_t = G_t - N_t \quad (15)$$

$$C_t = \min\{x_t, 5000/6, (10800 - S_t)/\eta_c\}, x_t \geq 0 \quad (16)$$

$$D_t = \min\{-x_t, 5000/6, \eta_d(S_t - 1200)\}, x_t < 0 \quad (17)$$

$$R_t = -x_t - D_t, x_t < 0 \quad (18)$$

富余分支中的放电量和紧急购电量为零，剩余富余电量弃置；缺口分支中的充电量和弃置量为零。该规则只使用当前状态与观测，能够满足物理约束，但可能提前消耗后续高价时段需要的储能，因此属于可执行的贪心策略。日末状态连续传递到下一日，不人为重置为 6000 kWh。

5.3.4 求解结果及分析

表 5 问题 2 的 334 天费用与电量

计算项目	数值结果
计划购电量 / kWh	21,282,449.77
计划购电费 / 元	13,206,734.13
紧急购电量 / kWh	224,803.88
紧急购电费 / 元	1,299,372.03
购电总费用 / 元	14,506,106.15

从费用构成可以看出，紧急购电虽然只占部分电量，却以 5 倍价格结算，对总费用有显著影响。扩大计划裕度可能减少缺口，但也会增加已付费富余，因此不能仅以紧急购电量最小作为计划目标。指定日期的购电、储能与紧急区间见 5.6 节，均来自实际连续回放轨迹。费用项与表 5 对应。

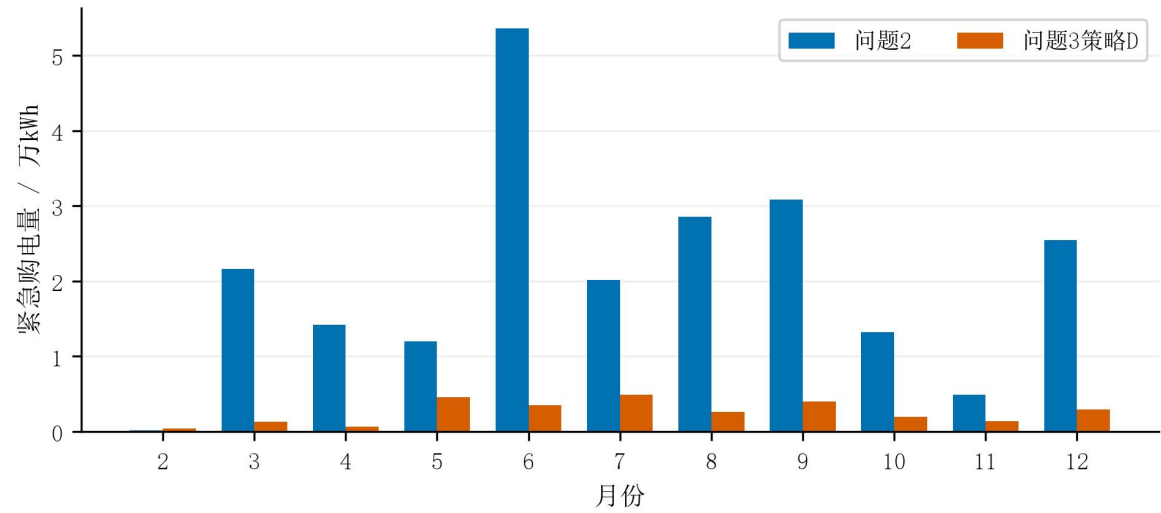


图 6 问题 2 与问题 3 策略 D 的月度紧急购电量

月度统计将全年紧急购电分解到各月，用于观察风险在时间上的分布及预报更新后的变化。两组策略均按同一实际序列连续运行，图中差异反映预测输入、计划调整和储能状态传递的综合作用。参见图 6。

5.4 问题 3 的模型建立与求解

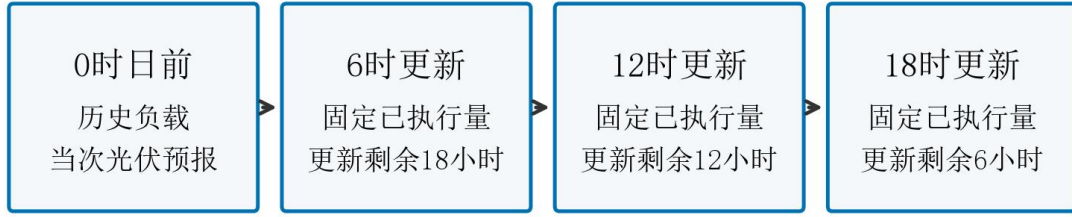


图 7 各预报时刻可调整的剩余时域

5.4.1 分时预报更新与剩余时域

附件 3 每次发布 24 个未来整点预报。本文以发布时间已经观测到的光伏功率作为左端锚点，在线性插值后提取当天未执行时段，并使用相同发布时间的历史预报误差构造残差情景。该处理区分了日前误差与临近预报误差。参见图 7。

负载没有同步发布的外部预报，因此用最近 1 小时已实现的残差均值修正日前预测，并按 6 小时尺度衰减。设发布时间为日内第若干小时，修正后的负载预测为：

$$\hat{L}_t^{\tau} = \max\{0, \hat{L}_t^0 + \bar{e}_{\tau} \exp[-(t - 6\tau)/36]\} \quad (19)$$

其中 τ 以小时计、 t 为 10 分钟时段索引。更新时以实际储电量作为剩余模型的初始状态，冻结所有已执行交易，只优化后续计划。本文借鉴滚动时域思想[4]，实施 6 小时预报触发的购电调整，电池每 10 分钟继续按实时规则执行。

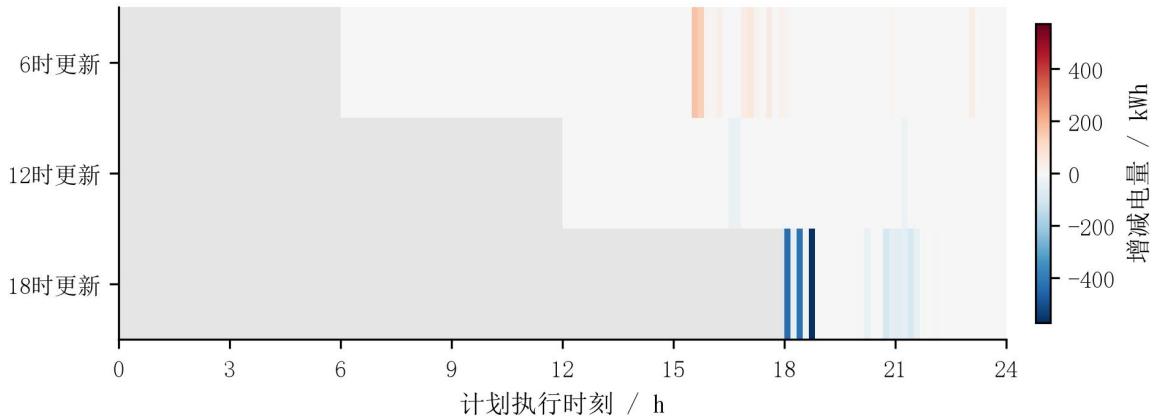


图 8 2025 年 6 月 21 日各次预报触发的计划增减

热力图中的红色表示相对上一有效计划增购，蓝色表示减购，灰色为更新时刻以前已经执行的区间。图中各次调整仅作用于未来时段，直观体现了已执行交易冻结的约束。参见图 8。

5.4.2 调整费用模型

每次调整以前有效计划为基准，将差额分为增购量和减购量，避免重复计费。按模型假设，增购按 1.5 倍电价结算；取消部分退回原款并收取 50% 违约金，因此其净退款为原价的 50%。

$$\delta_t^{k,+} = \max\{G_t^k - G_t^{k-1}, 0\}, \delta_t^{k,-} = \max\{G_t^{k-1} - G_t^k, 0\} \quad (20)$$

$$J_3 = \sum_t p_t G_t^0 + \sum_{k,t} (1.5 p_t \delta_t^{k,+} - 0.5 p_t \delta_t^{k,-}) + \sum_t 5 p_t R_t \quad (21)$$

$$G_t^{\text{final}} = G_t^0 + \sum_k \delta_t^{k,+} - \sum_k \delta_t^{k,-} \quad (22)$$

将最终常规购电量代入后，总费用还可写成最终常规电费、纯调整附加费与紧急购电费之和：

$$J_3 = \sum_t p_t G_t^{\text{final}} + 0.5 \sum_{k,t} p_t (\delta_t^{k,+} + \delta_t^{k,-}) + \sum_t 5 p_t R_t \quad (23)$$

两种写法应给出相同费用。净调整费可能为负，因为其中包含取消交易的退款；纯调整附加费则非负。费用核算时必须保留这一区别。

5.4.3 求解步骤与更新时点比较

Step1: 制定零点计划

依据 0 时光伏预报和历史负载预测构造情景，求解当天购电计划，并执行到下一预报时刻。

Step2: 更新状态与剩余计划

收到新预报后，读取已实现的储电状态和负载残差；对尚未执行区间重新求解，记录相对上一有效计划的增购、减购及相应费用。

Step3: 完成日内执行与年度评价

持续进行 10 分钟储能调节和紧急补购，将日末状态传递到次日。分别回放只用 0 时、加入 6 时、再加入 12 时及 18 时的四组策略。

表 6 不同预报更新策略的比较

策略	预报时刻	总费用 / 元	紧急量 / kWh	净调整费 / 元
A	0	14,643,207.45	220,412.16	0.00
B	0、6	14,265,453.54	94,010.96	389,089.89
C	0、6、12	14,083,138.95	66,497.09	383,298.59
D	0、6、12、18	13,968,861.90	28,257.01	453,886.70

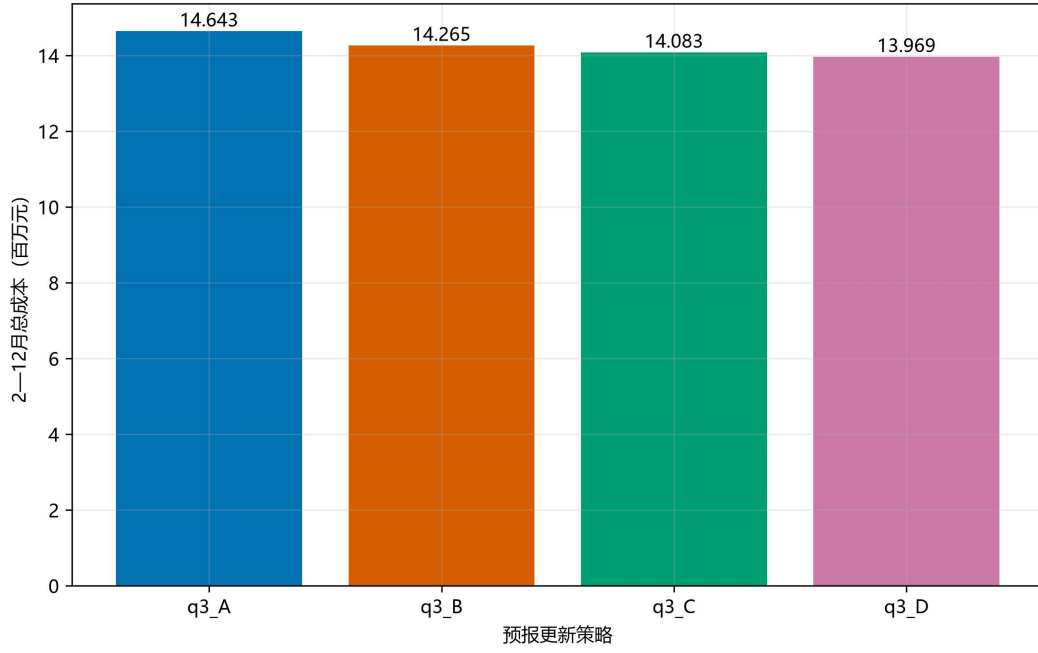


图 9 预报更新策略的费用比较

由 A 到 B、由 B 到 C、由 C 到 D，总费用依次减少 377753.91 元、182314.59 元和 114277.05 元。采用全部预报的策略 D 相对 A 节省 4.61%，紧急购电量降低 87.18%。因此，在现有数据与结算约定下，保留 6、12、18 时更新具有经济依据。参见图 9。

上述比较存在状态传递和策略联动，不能解释为每个时点独立的信息价值。对于题面以外的新增时刻，还需取得该时刻实际可用的预报，比较新增调整费、预测改善及储能机会成本。现有结果不能证明任意提高更新频率都会降低费用。

5.5 问题 4 的模型建立与求解

5.5.1 波动价格的信息条件

问题 4 保留前三问的能量和交易约束，将固定分时价格替换为附件 4 的价格序列。为避免未来价格泄漏，本文设置两类方案：一类假定当天曲线已知，作为与既有结果表对应的条件基准；另一类以前 7 天同时间价格均值预测当天价格，作为历史信息方案。

$$\hat{p}_{d,t} = \frac{1}{\min(7, d)} \sum_{j=\max(0, d-7)}^{d-1} p_{j,t}, d \geq 1 \quad (24)$$

第一天采用附件 1 电价初始化。历史信息方案在各次日内更新中继续使用 0 时生成的价格预测，不读取后续实际价格；计划优化使用预测价，账单统一按交易时刻的实际价计算。其他物理边界和储能执行规则保持一致，以便比较。

5.5.2 求解结果及分析

表 7 波动电价下的购电结果

策略	价格信息	总费用 / 元	紧急量 / kWh
4-2	当日曲线已知	15,249,408.93	237,748.18
4-3	当日曲线已知	14,608,808.73	36,078.78
4-2 扩展	历史价格预测	15,319,637.30	230,283.12
4-3 扩展	历史价格预测	14,732,259.05	34,167.83

当日曲线已知时，滚动调整方案相对日前方案降低费用 4.20%；仅利用历史价格预测时，对应降幅为 3.83%。两类条件下，利用额外光伏预报均在本次回放中降低了费用。参见图 11。

历史价格预测使 4-2 和 4-3 费用分别比条件基准高 0.46%和 0.85%。这一差额是给定预测与贪心执行规则下的经验结果，不是严格的信息价值上界。已知价格方案不能直接作为未知未来价格条件下的可实施最优策略。

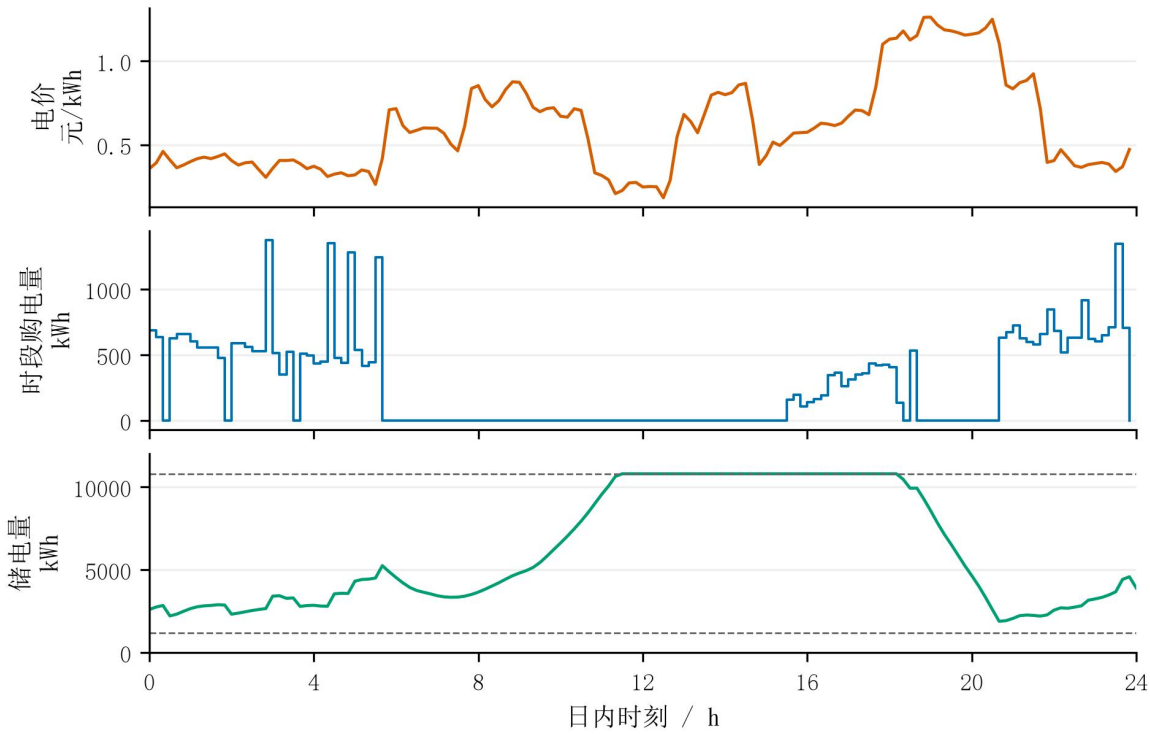


图 10 2025 年 6 月 21 日波动电价下的购电与储能联动

图示对应当天价格曲线已知条件下的问题 4-3 策略 D。共同时间轴展示价格变化、时段购电量与储电状态的联系；储电量虚线为上下边界。调度还受光伏、负载与调整费影响，不能仅凭价格高低判断某时段是否应购电。参见图 10。

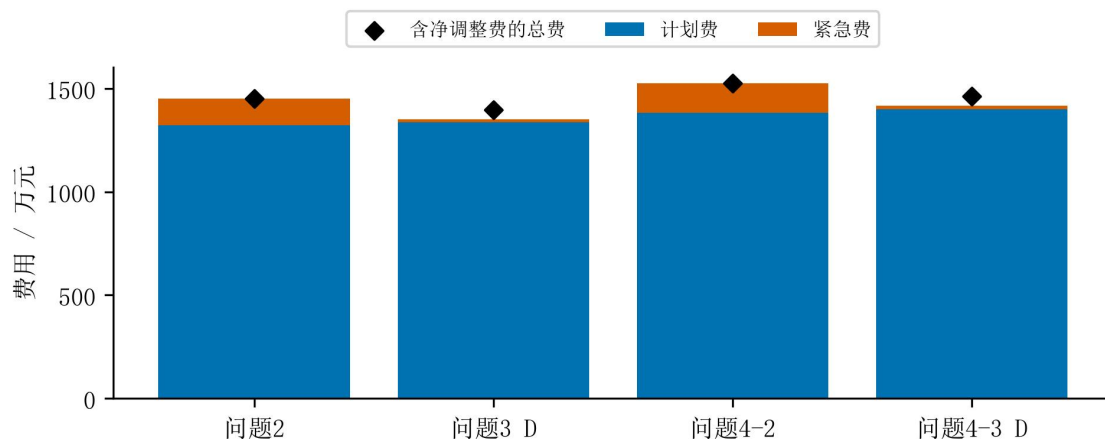


图 11 固定与波动电价下的费用构成

5.6 指定日期的调度结果

本节按题设表 1、表 2 与表 3 的结构给出 3 月 20 日、6 月 21 日、9 月 23 日和 12 月 21 日的结果。各表电量单位为 kWh，费用单位为元；日前计划量与全天计划费不含紧急购电，实际总费用另行列示。问题 3 采用策略 D；问题 4 的明细对应当日价格曲线已知的条件基准，历史价格预测方案汇总见 5.5 节。

5.6.1 问题 2 的指定日期结果

表 8 2025-03-20 日前购电量及全天结果

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.0000	12:00-12:10	771.3945	14:00-14:10	0.0000
16:00-16:10	461.5890	18:00-18:10	695.3051	20:00-20:10	0.0000
全天购电量	66,797.9104	全天购电费	39,990.10		

表 9 2025-03-20 实际储能充放电量

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	8,550.4921	406.8213	04:00-08:00	1,996.3357	5,894.0913
08:00-12:00	6,268.6252	2,520.6772	12:00-16:00	4,640.2298	684.5976
16:00-20:00	167.7293	6,084.6602	20:00-24:00	365.9685	2,353.2793
0:00 储电量	1,509.6215		24:00 储电量	1,362.1453	

表 10 2025-06-21 日前购电量及全天结果

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.0000	12:00-12:10	0.0000	14:00-14:10	0.0000
16:00-16:10	243.6957	18:00-18:10	429.7072	20:00-20:10	0.0000
全天购电量	38,371.2373	全天购电费	23,723.39		

表 11 2025-06-21 实际储能充放电量

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	1,773.2305	45.9992	04:00-08:00	722.6568	2,622.4424
08:00-12:00	6,801.7663	0.0000	12:00-16:00	0.0000	0.0000
16:00-20:00	17.6848	4,209.7206	20:00-24:00	2,621.9394	2,418.8181
0:00 储电量	5,397.0469		24:00 储电量	5,810.6188	

表 12 2025-09-23 日前购电量及全天结果

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.0000	12:00-12:10	458.3946	14:00-14:10	0.0000
16:00-16:10	514.8372	18:00-18:10	639.0266	20:00-20:10	0.0000
全天购电量	69,189.7195	全天购电费	43,401.10		

表 13 2025-09-23 实际储能充放电量

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	9,496.4946	18.1704	04:00-08:00	1,536.4526	5,269.0828
08:00-12:00	4,752.3260	1,896.6157	12:00-16:00	4,139.6910	575.6187
16:00-20:00	136.1602	6,010.1989	20:00-24:00	668.0329	2,676.1702
0:00 储电量	1,334.9402		24:00 储电量	1,718.0078	

表 14 2025-12-21 日前购电量及全天结果

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.0000	12:00-12:10	1,110.7036	14:00-14:10	0.0000
16:00-16:10	794.0558	18:00-18:10	684.0892	20:00-20:10	0.0000
全天购电量	100,381.7467	全天购电费	64,356.26		

表 15 2025-12-21 实际储能充放电量

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	9,004.0027	70.9169	04:00-08:00	1,799.9906	1,461.9493
08:00-12:00	5,667.3219	7,064.8587	12:00-16:00	6,743.7069	2,203.0187
16:00-20:00	85.1565	6,001.5592	20:00-24:00	613.3872	2,737.6507
0:00 储电量	1,892.1132		24:00 储电量	1,703.2630	

表 16 四个指定日期的紧急购电明细

03-20 时间段	03-20 购电量	06-21 时间段	06-21 购电量	09-23 时间段	09-23 购电量	12-21 时间段	12-21 购电量
09:10-10:00	256.7000	无	0.0000	21:00-21:50	407.3285	20:30-20:40	148.7820
20:20-21:50	952.9489	—	—	—	—	—	—

表 17 指定日期的完整费用核算

日期	计划费	净调整费	紧急费	总费用
2025-03-20	39,990.10	0.00	7,374.85	47,364.95
2025-06-21	23,723.39	0.00	0.00	23,723.39
2025-09-23	43,401.10	0.00	1,863.05	45,264.15
2025-12-21	64,356.26	0.00	1,037.90	65,394.16

同一四小时区间内充、放电累计量均为正，表示不同 10 分钟时段之间发生运行切换，并不表示同一时刻同时充放电。紧急区间由相邻正值时段合并，空余位置以横线表示。

5.6.2 问题 3 的指定日期结果

表 18 2025-03-20 日前购电量及全天结果

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.0000	12:00-12:10	667.6990	14:00-14:10	0.0000
16:00-16:10	614.4013	18:00-18:10	683.5314	20:00-20:10	0.0000
全天购电量	68,546.7593	全天购电费	41,927.74		

表 19 2025-03-20 实际储能充放电量

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	7,901.5416	312.4061	04:00-08:00	1,936.1898	5,992.9425
08:00-12:00	4,286.4608	2,565.1168	12:00-16:00	6,656.1390	294.4557
16:00-20:00	174.5376	5,709.0510	20:00-24:00	586.5721	3,044.0926
0:00 储电量	2,202.1144		24:00 储电量	1,680.4504	

表 20 2025-06-21 日前购电量及全天结果

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.0000	12:00-12:10	0.0000	14:00-14:10	0.0000
16:00-16:10	124.7031	18:00-18:10	421.8987	20:00-20:10	0.0000
全天购电量	39,195.5367	全天购电费	23,534.86		

表 21 2025-06-21 实际储能充放电量

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	2,025.1162	0.0000	04:00-08:00	1,380.7565	1,799.5612
08:00-12:00	7,883.0107	0.0000	12:00-16:00	0.0000	0.0000
16:00-20:00	3.3184	5,533.8769	20:00-24:00	2,357.3932	2,506.6821
0:00 储电量	2,639.5173		24:00 储电量	3,990.6860	

表 22 2025-09-23 日前购电量及全天结果

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.0000	12:00-12:10	439.0507	14:00-14:10	0.0000
16:00-16:10	582.8025	18:00-18:10	676.0235	20:00-20:10	0.0000
全天购电量	72,730.8991	全天购电费	45,783.96		

表 23 2025-09-23 实际储能充放电量

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	8,522.5849	76.4939	04:00-08:00	1,864.4549	6,721.1732
08:00-12:00	5,898.2149	1,896.6157	12:00-16:00	4,937.7984	412.1953
16:00-20:00	182.4415	5,975.3091	20:00-24:00	925.2721	2,805.0885
0:00 储电量	1,808.0385		24:00 储电量	2,031.4222	

表 24 2025-12-21 日前购电量及全天结果

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.0000	12:00-12:10	952.2926	14:00-14:10	0.0000
16:00-16:10	774.1051	18:00-18:10	730.2102	20:00-20:10	0.0000
全天购电量	97,742.2025	全天购电费	62,649.23		

表 25 2025-12-21 实际储能充放电量

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	9,014.6331	42.3126	04:00-08:00	1,882.7950	1,669.3056
08:00-12:00	5,494.4980	7,236.4164	12:00-16:00	7,333.5369	2,306.5261
16:00-20:00	20.6683	5,541.6255	20:00-24:00	444.7648	2,994.5754
0:00 储电量	1,952.1507		24:00 储电量	1,734.2221	

表 26 四个指定日期的紧急购电明细

03-20 时间段	03-20 购电量	06-21 时间段	06-21 购电量	09-23 时间段	09-23 购电量	12-21 时间段	12-21 购电量
09:10-10:00	212.2604	无	0.0000	20:20-20:30	6.3750	07:50-08:00	12.0105
—	—	—	—	21:20-21:50	138.4320	—	—

表 27 指定日期的完整费用核算

日期	计划费	净调整费	紧急费	总费用
2025-03-20	41,927.74	3,100.82	1,029.39	46,057.96
2025-06-21	23,534.86	-449.60	0.00	23,085.25
2025-09-23	45,783.96	-1,483.70	678.98	44,979.24
2025-12-21	62,649.23	1,781.88	69.76	64,500.87

表 28 调整后的最终常规购电量

日期	10:00	12:00	14:00	16:00	18:00	20:00	全天
03-20	0.00	530.09	0.00	524.60	657.37	0.00	66,290.74
06-21	0.00	0.00	0.00	142.42	0.00	0.00	37,757.00
09-23	0.00	320.23	0.00	520.84	676.02	3.63	67,830.15
12-21	0.00	952.80	0.00	852.73	730.21	0.00	97,256.84

上表各整点列表示该时刻开始的 10 分钟购电量，展示精度为两位小数，完整精度见结果表。“最终常规购电”已包含各次调整，不能再与增购量重复相加。

同一四小时区间内充、放电累计量均为正，表示不同 10 分钟时段之间发生运行切换，并不表示同一时刻同时充放电。紧急区间由相邻正值时段合并，空余位置以横线表示。

5.6.3 问题 4-2 的指定日期结果

表 29 2025-03-20 日前购电量及全天结果

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.0000	12:00-12:10	771.3945	14:00-14:10	0.0000
16:00-16:10	461.5890	18:00-18:10	0.0000	20:00-20:10	0.0000
全天购电量	67,716.3431	全天购电费	41,113.08		

表 30 2025-03-20 实际储能充放电量

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	6,706.3072	512.6354	04:00-08:00	3,753.2098	5,752.8951
08:00-12:00	6,268.6252	2,548.0120	12:00-16:00	4,632.0511	719.4601
16:00-20:00	144.4381	6,685.6107	20:00-24:00	1,868.8262	2,214.3473
0:00 储电量	1,579.2597		24:00 储电量	2,134.3043	

表 31 2025-06-21 日前购电量及全天结果

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.0000	12:00-12:10	0.0000	14:00-14:10	0.0000
16:00-16:10	243.6957	18:00-18:10	429.7072	20:00-20:10	0.0000
全天购电量	39,414.3625	全天购电费	20,287.91		

表 32 2025-06-21 实际储能充放电量

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	1,982.4727	3,695.4480	04:00-08:00	3,305.5459	1,719.9540
08:00-12:00	7,621.6876	0.0000	12:00-16:00	0.0000	0.0000
16:00-20:00	15.6893	4,170.5899	20:00-24:00	3,258.6098	3,117.4407
0:00 储电量	5,198.3777		24:00 储电量	5,649.0575	

表 33 2025-09-23 日前购电量及全天结果

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.0000	12:00-12:10	458.3946	14:00-14:10	0.0000
16:00-16:10	514.8372	18:00-18:10	0.0000	20:00-20:10	739.2926
全天购电量	69,181.2047	全天购电费	44,589.89		

表 34 2025-09-23 实际储能充放电量

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	6,510.9961	18.2573	04:00-08:00	4,481.1549	5,232.7774
08:00-12:00	4,825.7005	1,896.6157	12:00-16:00	4,184.4661	693.7682
16:00-20:00	254.9077	6,868.2389	20:00-24:00	791.2947	2,027.5618
0:00 储电量	1,331.4140		24:00 储电量	1,678.1716	

表 35 2025-12-21 日前购电量及全天结果

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.0000	12:00-12:10	1,110.7036	14:00-14:10	0.0000
16:00-16:10	794.0558	18:00-18:10	684.0892	20:00-20:10	0.0000
全天购电量	99,075.3136	全天购电费	75,314.29		

表 36 2025-12-21 实际储能充放电量

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	5,904.9303	70.9169	04:00-08:00	806.9479	1,452.3576
08:00-12:00	5,808.2857	7,205.8225	12:00-16:00	7,086.1338	2,487.4074
16:00-20:00	126.8923	5,314.2748	20:00-24:00	3,704.2350	2,249.9071
0:00 储电量	5,564.3592		24:00 储电量	5,790.6127	

表 37 四个指定日期的紧急购电明细

03-20 时间段	03-20 购电量	06-21 时间段	06-21 购电量	09-23 时间段	09-23 购电量	12-21 时间段	12-21 购电量
09:10-10:00	229.3652	无	0.0000	20:30-20:40	9.4001	无	0.0000
20:20-21:50	965.9920	—	—	20:50-21:50	442.9388	—	—

表 38 指定日期的完整费用核算

日期	计划费	净调整费	紧急费	总费用
2025-03-20	41,113.08	0.00	7,139.39	48,252.47
2025-06-21	20,287.91	0.00	0.00	20,287.91
2025-09-23	44,589.89	0.00	2,221.80	46,811.69
2025-12-21	75,314.29	0.00	0.00	75,314.29

同一四小时区间内充、放电累计量均为正，表示不同 10 分钟时段之间发生运行切换，并不表示同一时刻同时充放电。紧急区间由相邻正值时段合并，空余位置以横线表示。

5.6.4 问题 4-3 的指定日期结果

表 39 2025-03-20 日前购电量及全天结果

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.0000	12:00-12:10	667.6990	14:00-14:10	0.0000
16:00-16:10	614.4013	18:00-18:10	0.0000	20:00-20:10	0.0000
全天购电量	69,440.3509	全天购电费	43,015.62		

表 40 2025-03-20 实际储能充放电量

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	6,239.7904	504.1176	04:00-08:00	3,430.3878	5,757.7905
08:00-12:00	4,283.9900	2,575.5103	12:00-16:00	6,658.6098	294.4557
16:00-20:00	174.5376	6,377.8626	20:00-24:00	1,992.3802	2,899.4282
0:00 储电量	2,316.1934		24:00 储电量	2,363.2918	

表 41 2025-06-21 日前购电量及全天结果

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.0000	12:00-12:10	0.0000	14:00-14:10	0.0000
16:00-16:10	128.0093	18:00-18:10	438.5881	20:00-20:10	0.0000
全天购电量	39,781.5323	全天购电费	20,452.48		

表 42 2025-06-21 实际储能充放电量

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	2,368.6485	1,687.5172	04:00-08:00	3,079.7811	1,778.2734
08:00-12:00	7,943.5653	0.0000	12:00-16:00	0.0000	0.0000
16:00-20:00	2.7747	5,537.9433	20:00-24:00	3,094.9299	3,177.7014
0:00 储电量	2,598.0832		24:00 储电量	3,903.8845	

表 43 2025-09-23 日前购电量及全天结果

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.0000	12:00-12:10	439.0507	14:00-14:10	0.0000
16:00-16:10	582.8025	18:00-18:10	0.0000	20:00-20:10	805.4434
全天购电量	72,731.6962	全天购电费	47,105.84		

表 44 2025-09-23 实际储能充放电量

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	6,212.4251	165.5270	04:00-08:00	4,254.0343	6,680.0303
08:00-12:00	5,403.7451	1,896.6157	12:00-16:00	5,426.0405	412.1953
16:00-20:00	269.1458	6,888.5569	20:00-24:00	935.5478	1,979.8807
0:00 储电量	1,795.3772		24:00 储电量	2,020.8820	

表 45 2025-12-21 日前购电量及全天结果

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.0000	12:00-12:10	952.2926	14:00-14:10	0.0000
16:00-16:10	774.1051	18:00-18:10	730.2102	20:00-20:10	0.0000
全天购电量	97,555.7323	全天购电费	73,293.83		

表 46 2025-12-21 实际储能充放电量

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	5,763.9579	74.6913	04:00-08:00	779.2716	1,478.9757
08:00-12:00	5,467.7745	7,305.0077	12:00-16:00	7,651.3700	2,473.7335
16:00-20:00	0.0000	5,388.0558	20:00-24:00	4,353.3056	3,058.6478
0:00 储电量	5,695.4283		24:00 储电量	5,332.7488	

表 47 四个指定日期的紧急购电明细

03-20 时间段	03-20 购电量	06-21 时间段	06-21 购电量	09-23 时间段	09-23 购电量	12-21 时间段	12-21 购电量
09:10-10:00	201.8669	无	0.0000	18:20-18:30	1.0833	07:50-08:00	12.0105
—	—	—	—	20:20-20:30	6.3750	—	—
—	—	—	—	21:20-21:50	146.6475	—	—

表 48 指定日期的完整费用核算

日期	计划费	净调整费	紧急费	总费用
2025-03-20	43,015.62	3,247.08	1,024.43	47,287.13
2025-06-21	20,452.48	-556.59	0.00	19,895.89
2025-09-23	47,105.84	-1,569.54	775.46	46,311.76
2025-12-21	73,293.83	1,566.88	84.45	74,945.16

表 49 调整后的最终常规购电量

日期	10:00	12:00	14:00	16:00	18:00	20:00	全天
03-20	0.00	530.09	0.00	524.60	0.00	0.00	67,169.02
06-21	0.00	0.00	0.00	142.42	406.97	0.00	38,346.92
09-23	0.00	324.72	0.00	520.84	0.00	756.57	67,663.51
12-21	0.00	952.80	0.00	852.73	683.52	0.00	96,887.34

上表各整点列表示该时刻开始的 10 分钟购电量，展示精度为两位小数，完整精度见结果表。“最终常规购电”已包含各次调整，不能再与增购量重复相加。

同一四小时区间内充、放电累计量均为正，表示不同 10 分钟时段之间发生运行切换，并不表示同一时刻同时充放电。紧急区间由相邻正值时段合并，空余位置以横线表示。

5.7 模型检验

检验分为灵敏度、稳健性、误差和对比验证四个方面。灵敏度改变模型参数，稳健性考察已定计划在不利输入下的表现，误差分析评价预测偏离程度，对比验证核对求解正确性及策略收益。四类检验使用不同证据，避免重复解释同一指标。参见图 12。

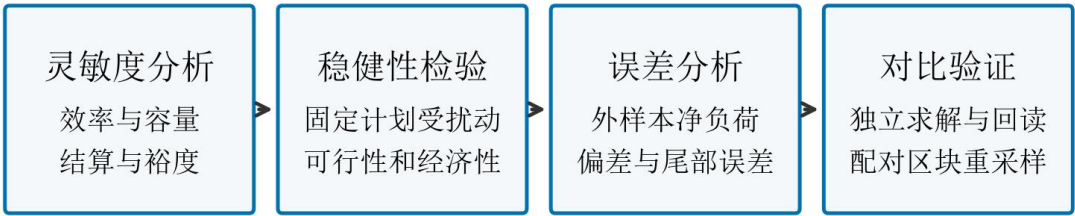


图 12 四方面模型检验框架

5.7.1 灵敏度分析

选取 3 月 18—23 日、6 月 19—24 日、9 月 21—26 日及 12 月 19—24 日，共 24 天进行对比。各组采用相同初始储电量，并用库存价值校正终端状态差异：

$$J_{\text{adj}} = J - 0.5\bar{p}(S_{\text{end}} - S_{\text{start}})$$

(25)

表 50 问题 3 策略 D 的参数敏感性

参数情形	库存校正费用 / 元	相对基准变化
基准参数	1,022,316.93	+0.00%
充放电效率各 85%	1,061,720.06	+3.85%
充放电效率各 95%	987,555.12	-3.40%
往返效率 90%	988,531.27	-3.30%
容量 9600 kWh	1,072,606.20	+4.92%
容量 14400 kWh	978,885.00	-4.25%
残差裕度 1	1,093,889.71	+7.00%
取消不退款	1,030,790.03	+0.83%
PCHIP 插值	1,022,705.50	+0.04%

在所测范围内，提高效率与扩大容量能够降低费用；增加安全裕度可能因额外购电和弃置而提高成本。取消不退款时，策略 D 费用上升，但未在该条款下重新比较全部 A—D 组合，因此不能断言所有更新时点仍然有利。24 天结果用于说明局部变化趋势，不外推为全年收益保证。

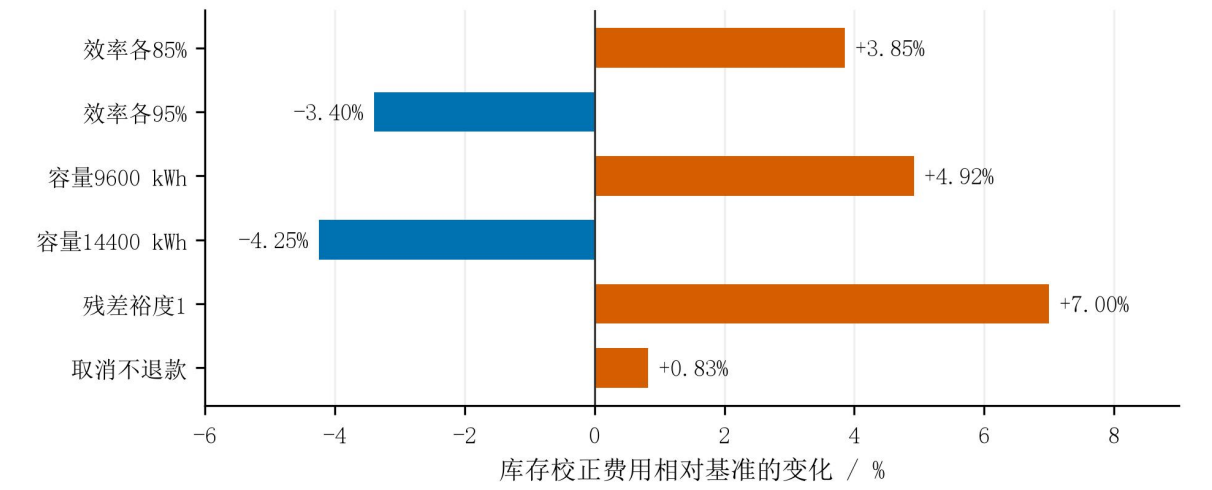


图 13 关键参数变化对 24 日库存校正费用的影响

以零变化线为参照，左侧表示费用下降，右侧表示费用上升。该图与敏感性表使用相同的策略 D 和 24 日样本，便于比较效率、容量及结算参数的影响方向，不能据此代替全年重新求解。参见图 13。

5.7.2 稳健性检验

将问题 3 策略 D 的 334 天常规购电轨迹固定，保持 2 月初实际库存，分别施加负载增加 5%、光伏减少 10%及二者同时发生的持续扰动。全过程重新执行电池控制和紧急补购，日末库存连续传递；未来观测不用于改写计划。本试验衡量既定计划的抗扰能力，不能解释为允许重新预测和调度后的最优费用。

表 51 固定计划的持续扰动检验

扰动情形	紧急电量 / 万 kWh	紧急费用 / 万元
原始负荷	2.83	14.48
负载增加 5%	113.07	664.42
光伏减少 10%	98.09	623.39
联合不利扰动	242.88	1,470.09

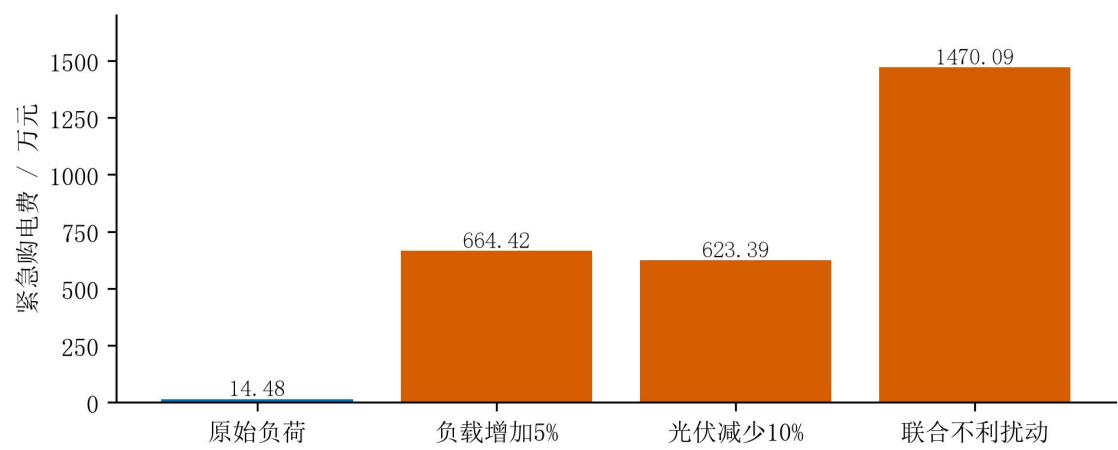


图 14 固定常规购电计划在持续不利扰动下的代价

四种情形的储电量均保持在 1200—10800 kWh，平衡误差不超过 1.14×10^{-13} kWh。联合扰动使紧急费用由 14.48 万元增加至 1470.09 万元，表明供电可行性具有保障，但既定计划的经济稳健性不足。该保障依赖题目未设外网购电上限；实际系统若存在外网限额，必须另设负荷损失约束和应急机制。参见图 14。

与此不同，灵敏度实验中的残差裕度增加和不退款情形会重新优化计划，检验的是模型决策对参数变化的响应。两类试验的决策是否允许调整不同，费用不能混合排序。

5.7.3 误差分析

以 2 月至 12 月共 48096 个 10 分钟时段为外样本，对岭回归的净负荷功率预测计算误差。误差定义为预测减实际，正值意味着计划输入偏高，负值意味着低估用电需求。采用 MAE、RMSE、平均偏差和绝对误差 95%分位数共同描述典型偏差与尾部偏差。

$$e_i = \hat{N}_i^P - N_i^P, \text{MAE} = \frac{1}{n} \sum_i |e_i|, \text{RMSE} = \sqrt{\frac{1}{n} \sum_i e_i^2} \quad (26)$$

表 52 净负荷功率预测的外样本误差

指标	MAE	RMSE	平均偏差	绝对误差 95%分位
数值 / kW	272.78	387.57	-13.66	832.61

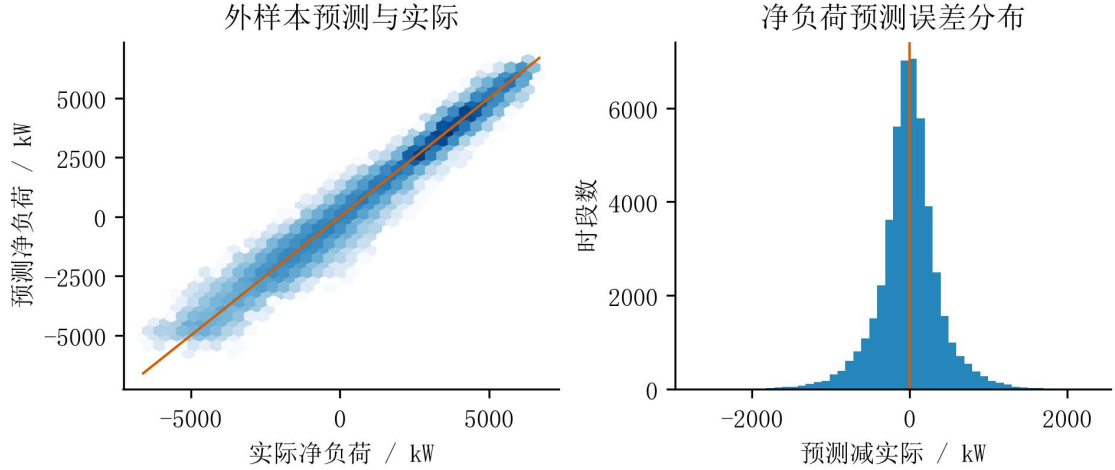


图 15 净负荷预测与实际的分布及预测误差

平均偏差为-13.66 kW，存在轻微低估；RMSE 高于 MAE，表明少量大误差会放大二次损失。绝对误差 95%分位为 832.61 kW，对应单时段 138.77 kWh，仅用于解释量级，不能作为保证覆盖 95%未来需求的储能容量。净负荷可能接近零，故不采用其 MAPE 作为主指标。该误差针对日前岭回归，不等同于更新后的光伏预报误差。参见图 15。

5.7.4 对比验证

首先进行数值与物理对比。问题 1 由独立原始与对偶模型、双单纯形和内点法验证，目标差约 7.28×10^{-12} 元。12 组全年策略共 132 项物理与费用检查的最大误差为 5.82×10^{-11} ；五份工作簿的 288720 个购电时段值独立回读，最大差异约 1.2×10^{-10} 。另有 61 项未来信息污染测试支持已检查路径的信息边界。

其次比较相同信息条件下的问题 3 策略 A 与 D。逐日费用先按相同价格系数校正日初日末库存，再计算配对节省；以 7 日连续区块作 2000 次移动区块重采样，固定随机种子 20260912。这样保留部分周内相关性，避免把 48096 个时段当作相互独立样本。

$$\Delta J_d = J_{A,d}^{\text{adj}} - J_{D,d}^{\text{adj}} \quad (27)$$

策略 D 在 334 天中的 190 天实现正节省，日均库存校正节省为 2,019.08 元；重采样均值的 2.5%和 97.5%分位为 1,150.08 元和 3,183.14 元。该区间是固定年度样本与 7

日区块设定下的经验不确定性范围；季节非平稳性、区块长度和仅一年数据限制其外推，不作跨年份收益保证。参见图 16。

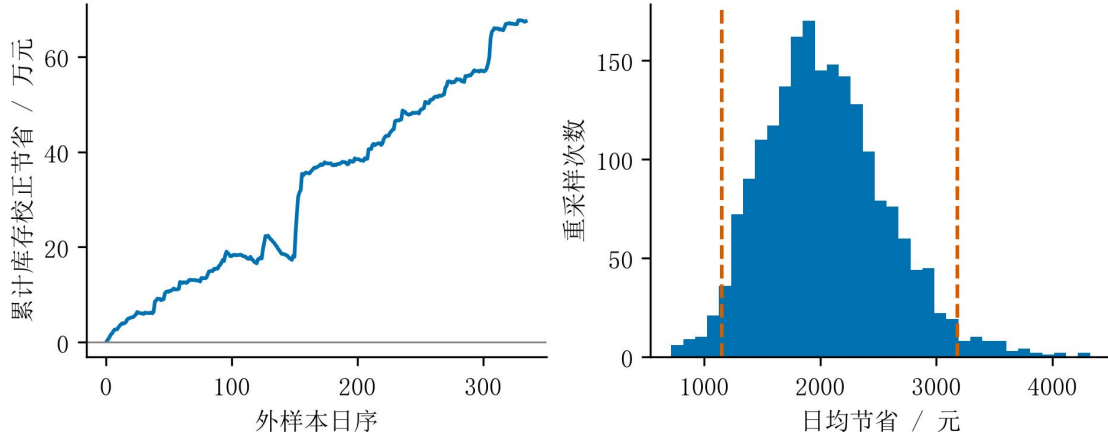


图 16 策略 D 相对 A 的配对节省及区块重采样分布

5.8 模型深化与扩展验证

为抑制代表情景中的大额紧急购电，在原期望费用模型中引入最坏情景损失上界。设各情景紧急费用为损失，以非负权重控制其附加惩罚：

$$\min J_{\lambda} = J_{\text{base}} + \lambda Z, Z \geq \sum_t 5 p_t R_t^{(s)}, s = 1, \dots, K \quad (28)$$

新增变量和约束均为线性，因此仍可采用原稀疏线性规划求解器。权重候选为 0、0.1、0.3 和 1，仅在 1 月 17—31 日选择，再在四季 24 日验证；这是一项针对尾部成本的模型扩展，不改变已有结果表的信息条件。

实测四种权重在 1 月的库存校正费用均为 936469.26 元，四季 24 日均为 1022316.93 元，未取得可辨别的费用改善，因此按较简原则保留权重 0。仅提高现有三个情景的风险惩罚不足以修复未被情景覆盖的持续系统偏差；后续应优先扩充极端残差情景、校准情景权重，并引入带条件预测的实时储能优化。未验证改善不写入主方案收益。

第 6 章 模型评价与改进

6.1 模型的优点

本文以统一的能量平衡和储能状态连接四个问题，使功率、电量、库存与费用之间的关系清晰。确定性问题可通过线性规划得到最优解，并给出对偶检验；不确定性问题遵循信息发布时间进行连续回放，计划、调整和紧急购电分别计量，能够追溯指定日期结果。

模型将预测方法的经济选型与外样本评价分开，并同时报告波动电价的两种信息条件，有助于避免将事后已知数据误用于决策。对容量、效率及结算条款的敏感性检验，也使结论的适用范围更加明确。

6.2 模型的不足

三条历史残差情景不能完整刻画极端天气和尾部风险，情景内动作缺少完整的多阶段非预见性约束。实际电池采用贪心执行规则，没有显式优化未来储能机会成本。因此，问题 2 至问题 4 所得费用属于所构造策略的回放结果，不具有全局最优保证。

模型未考虑自放电、寿命损耗、售电和外网交易功率限制，结算规则也采用了明确的退款约定。这些简化与题目数据范围相适应，但推广到实际微网前需要补充设备与市场约束。

6.3 模型的改进方向

在已完成的风险惩罚扩展基础上，可将电池执行规则改进为基于条件预测的短时段优化，并在场景树中加入非预见性约束，使储能决策兼顾后续风险。对于新增价格发布机制、退款条款或预报时点，应重新训练和回放，通过统一信息边界、终端价值与实际账单比较评估收益。

综合结果，问题 1 的最低日购电费为 35,126.95 元；问题 2 与问题 3 策略 D 的 2 月至 12 月费用分别为 1,450.61 万元和 1,396.89 万元。已有三个额外预报时点在所测嵌套策略中均带来节省，波动价格下的结论则需结合价格信息条件解释。

人工智能工具使用声明

本研究在资料核对、程序审查、计算复核、模型扩展试验、图形绘制、论文文字修订及 Word 公式排版中使用 OpenAI Codex 辅助。建模所用数值输入为题设附件；正文中的新增数值来自所附程序的实际执行结果，未以生成式文本代替数值计算。

AI 参与范围包括提出检验方案、编写和调整 Python 程序、整理参考文献信息、转换 LaTeX 公式为 Word 数学对象、建立自动编号及交叉引用。AI 提出的风险惩罚扩展已作验证，未观察到收益，因此没有宣称主模型性能提高。工具输出与程序运行记录保存在本次工作目录及交互记录中。

参考文献

- [1] SCIPY DEVELOPERS. scipy.optimize.linprog[EB/OL]. [2026-09-12].
<https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.linprog.html>.
- [2] BOYD S, VANDENBERGHE L. Convex optimization[M]. Cambridge: Cambridge University Press, 2004.
- [3] HYNDMAN R J, ATHANASOPOULOS G. Forecasting: principles and practice[M/OL]. 3rd ed. Melbourne: OTexts, 2021[2026-09-12]. <https://otexts.com/fpp3/>.
- [4] RAWLINGS J B, MAYNE D Q, DIEHL M M. Model predictive control: theory, computation, and design[M/OL]. 2nd ed. Nob Hill Publishing, 2026[2026-09-12].
<https://sites.engineering.ucsb.edu/~jbrow/mpc/>.

附录 A 支撑材料与模块运行说明

程序按输入、预测、优化、执行、验证和绘图分层封装。TXT 文件保存完整 UTF-8 Python 源码，恢复脚本核对 SHA256 后生成同名 PY 模块；无需从 Word 复制长代码。原始附件放入 input，所有结果写入程序包内部目录。正文中的指定日期结果仍使用前版已经独立核验的完整轨迹。

表 53 程序包功能索引

TXT 文件	功能
common.txt	公共参数和文件接口
data_io.txt	附件解析与审计
forecasting.txt	因果预测与误差
optimization.txt	线性规划求解
dispatch.txt	运行与交易结算
validation.txt	物理与费用验证
model.txt	模块统一接口
run.txt	四问计算 参数选型 灵敏度实验
q1_certificate.txt	独立求解与原始对偶检验
extended_checks.txt	压力 误差 重采样及风险模型扩展
causality.txt	未来信息污染检验
make_figures.txt	新增八图复现
figures_legacy.txt	基础结果图形复现
figures_added.txt	前版六幅分析图复现

功能模块均为 UTF-8 TXT，内容是完整 Python 源代码。恢复脚本可直接由 Python 运行，无须手工改扩展名。

1. 安装 Python 3.12 及依赖：python -m pip install -r requirements.txt
2. 将附件 1.xlsx 至附件 4.xlsx 和附件 5 中的五份原始模板放在本目录 input 文件夹。
3. 运行 python restore_and_run.txt --smoke，核验附件解析和问题 1 两个独立求解器。
4. 运行 python restore_and_run.txt --all，执行全量预测选型、12 组全年回放、24 日敏感性、因果测试及新增检验。全量计算耗时明显长于 smoke。
5. 可单独运行 src/run.py --stage forecast、annual、sensitivity。强制重算全年用 annual_force。单独阶段须有前序结果。

6. `results` 保存全精度 NPZ 及 CSV；`audit` 保存 11 类约束和因果检查；计算结果保存问题 1 证书；检验结果保存新增检验。当前包不执行历史 Excel 导出实现，避免覆写原表。

7. 各模块功能、SHA256 与恢复后的文件名见文件清单.json。不得更改单个 TXT 后继续使用旧校验清单。

8. `figures_legacy` 和 `figures_added` 重现基础结果图及前版六图，`make_figures` 重现本次新增八图；问题 1 独立轨迹图由 `q1_certificate` 生成。

输入：题设附件 1 至 4 和附件 5 模板。中间结果：`results` 中的逐时段轨迹、逐日费用、预测缓存及灵敏度表。检验输出：`audit` 和检验结果。依赖：NumPy、SciPy、pandas、openpyxl、matplotlib。运行顺序为附件解析、预测选型、全年回放、灵敏度、独立校验和图形生成。基础计算与新增检验均保留独立入口。

附录 B 恢复入口与依赖清单

先放入原始附件，再执行恢复脚本。`smoke` 用于附件解析及问题 1 双求解器验证；`all` 执行完整管线。脚本仅向本包内写入，不删除工程原文件。

```
"""直接运行 python restore_and_run.txt --smoke; 只向本包的 src 和结果目录写入。"""
from pathlib import Path
import argparse, subprocess, sys, hashlib, json
root=Path(__file__).resolve().parent
parser=argparse.ArgumentParser();parser.add_argument('--
smoke',action='store_true');parser.add_argument('--all',action='store_true');args=parser.parse_args()
for folder in ['src','input','audit','results','计算结果','检验结果','图','figures']:
    (root/folder).mkdir(exist_ok=True)
manifest=json.loads((root/'文件清单.json').read_text(encoding='utf-8'))
for entry in manifest:
    raw=(root/entry['file']).read_bytes()
    assert hashlib.sha256(raw).hexdigest()==entry['sha256'],entry['file']
    (root/'src'/entry['module']).write_bytes(raw)
def run(name,*argv):subprocess.run([sys.executable,str(root/'src'/name),*argv],check=True)
if args.smoke:run('run.py','--stage','q1');run('q1_certificate.py')
if args.all:
    run('run.py','--
stage','all');run('q1_certificate.py');run('causality.py');run('extended_checks.py');run('make_figures.
py');run('figures_legacy.py');run('figures_added.py')
print('模块恢复及请求的运行阶段已完成。')
```

附录 C.1 公共参数和文件接口

源文件：common.txt；恢复文件：src/common.py。

```
"""公共参数和文件接口。输入输出单位沿用论文；发生约束错误时直接报错。"""
from pathlib import Path
import json,hashlib,datetime as dt
from functools import lru_cache
import numpy as np
import pandas as pd
from openpyxl import load_workbook
from scipy.optimize import linprog
from scipy.sparse import lil_matrix
from scipy.interpolate import PchipInterpolator
ROOT=Path(__file__).resolve().parents[1]
DT=1/6;T=144;SEED=20260910
DATES=pd.date_range('2025-01-01',periods=365)
METHODS=['mean','ma7','ma14','ma30','ewma','ridge']

def dump(obj,path):
    # 将NumPy 标量转为 JSON 可序列化数值；UTF-8 保存中文检查结果。
    Path(path).write_text(json.dumps(obj,ensure_ascii=False,indent=2,default=lambda x:x.item() if
    isinstance(x,np.generic) else str(x)),encoding='utf-8')

def time_label(t):
    # 输入 10 分钟时段索引，输出 24 小时制时间标签。
    return f'{t//6:02d}:{t%6*10:02d}'

def interval(t,end=None):return time_label(t)+'-'+time_label(t+1 if end is None else end)
    # 把左端及可选右端索引转换为区间标签。

def align_time(label):
    # 模板按区间左端匹配，末列解释为本日首时段；这是论文约定。
    # 原模板最后列统一解释为本日 00:00--00:10；其他列按左端精确匹配。
    left=str(label).split('-')[0].replace('+1','')
    h,m=map(int,left.split(':')[2]); return (h*60+m)//10

def _minute(v):
    # 把 Excel 时间或次日标记转换为分钟索引。
    if isinstance(v,dt.time):return v.hour*60+v.minute
    s=str(v);return 1440 if '+1' in s else int(s.split(':')[0])*60+int(s.split(':')[1])
```

附录 C.2 附件解析与审计

源文件：data_io.txt；恢复文件：src/data_io.py。

"""附件解析与审计。输入输出单位沿用论文；发生约束错误时直接报错。"""

```
from common import *
from common import _minute

def load_data():
    # 读取附件 1 至 4 及模板，验证形状、非负、有限值和日期顺序；生成数据审计和 NPZ 缓存。
    if not (ROOT/'input'/'附件 1.xlsx').exists():
        raise FileNotFoundError('请将附件 1 至 4 及附件 5 模板放入本复现目录的 input 文件夹')
    w1=load_workbook(ROOT/'input'/'附件 1.xlsx',data_only=True)
    rows=list(w1.active.values); a=np.asarray([r[1:4] for r in rows[1:]],float)
    assert [_minute(r[0]) for r in rows[1:]]==list(range(10,1441,10))
    def matrix(file,sheet=None):
        w=load_workbook(ROOT/'input'/file,data_only=True);s=w[sheet] if sheet else w.active
        r=list(s.values)
        assert [_minute(x) for x in r[0][1:]]==list(range(10,1441,10))
        assert list(pd.to_datetime([x[0] for x in r[1:]]))==list(DATES)
        return np.asarray([x[1:] for x in r[1:]],float)
    L=matrix('附件 2.xlsx','小区负载');PV=matrix('附件 2.xlsx','光伏发电实际功率');P=matrix('附件 4.xlsx')
    w3=load_workbook(ROOT/'input'/'附件 3.xlsx',data_only=True)
    F=np.full((365,4,24),np.nan);mapping=[];seen=set();last=None
    for row in list(w3.active.values)[1:]:
        if row[0] not in [None,'']:last=pd.Timestamp(row[0])
        hour=int(str(row[1]).split(':')[0]); day=(last-DATES[0]).days;assert hour in [0,6,12,18]
        assert (day,hour) not in seen;seen.add((day,hour));F[day,hour//6]=row[2:26]
        for k in
range(24):mapping.append([str(last+pd.Timedelta(hours=hour)),k+1,str(last+pd.Timedelta(hours=hour+k+1))
])
    assert len(seen)==1460
    pd.DataFrame(mapping,columns=['发布时间','预报第几小时','目标时间']).to_csv(ROOT/'audit'/'预报时间映
射.csv',index=False,encoding='utf-8-sig')
    stats=[]
    for name,v,unit in [('附件 1 电价',a[:,0],'元/kWh'),('附件 1 负载',a[:,1],'kW'),('附件 1 光伏
',a[:,2],'kW'),('附件 2 负载',L,'kW'),('附件 2 光伏',PV,'kW'),('附件 3 预报',F,'kW'),('附件 4 电价',P,'元
/kWh')]:
        stats.append(dict(数据=name,形状=str(v.shape),单位=unit,数量=v.size,缺失=int(np.isnan(v).sum()),
非有限=int((~np.isfinite(v)).sum()),负值=int((v<0).sum()),最小=float(v.min()),最大=float(v.max()),均值
=float(v.mean())))
        assert np.isfinite(v).all() and (v>=0).all(),name
    pd.DataFrame(stats).to_csv(ROOT/'audit'/'数据审计.csv',index=False,encoding='utf-8-sig')
    manifest={f.name:hashlib.sha256(f.read_bytes()).hexdigest() for f in (ROOT/'input').iterdir()}
    dump(manifest,ROOT/'audit'/'输入 SHA256.json')
    tm=[];templates={}
    for f in (ROOT/'input').glob('result*.xlsx'):
        w=load_workbook(f);s=w['计划购电量'];labels=[s.cell(i,1).value for i in range(2,146)] if
f.stem=='result1' else [s.cell(1,j).value for j in range(2,146)]
        ids=[align_time(x) for x in labels];assert sorted(ids)==list(range(144))
        for i,(label,t) in
enumerate(zip(labels,ids)):tm.append([f.name,i+1,label,interval(t),time_label(t+1),t,'原末列解释' if
t==0 else '按左端匹配'])
        templates[f.name]={s.title:[s.max_row,s.max_column] for s in w}
    pd.DataFrame(tm,columns=['文件','数据序号','原模板标签','规范区间','附件右端采样时刻','内部索引','说明
']).to_csv(ROOT/'audit'/'模板逐列时间映射.csv',index=False,encoding='utf-8-sig')
    dump(templates,ROOT/'audit'/'模板结构.json')
    np.savez_compressed(ROOT/'results'/'data.npz',a=a,L=L,PV=PV,P=P,F=F)
    return dict(a=a,L=L,PV=PV,P=P,F=F)
```

附录 C.3 因果预测与误差

源文件：forecasting.txt；恢复文件：src/forecasting.py。

```
"""因果预测与误差。输入输出单位沿用论文；发生约束错误时直接报错。"""
from common import *
from data_io import *
from common import _minute

def forecast(y,d,method,fallback):
    # 仅以目标日前的样本预测；参数method选择六种方法，fallback处理无历史情形。返回144个功率值。
    """只能访问 y[:d]；不同候选均遵循相同截止点。"""
    if d==0:return fallback.copy()
    h=y[:d]
    if method=='mean':v=h.mean(axis=0)
    elif method.startswith('ma'):v=h[-int(method[2:]):].mean(axis=0)
    elif method=='ewma':
        weight=.8*np.arange(min(d,30)-1,-1,-1);v=np.average(h[-30:],axis=0,weights=weight)
    else:
        if d<12:return forecast(y,d,'ma7',fallback)
        def feat(j):
            return np.stack([np.ones(T),y[j-1]/5000,y[j-7]/5000,y[max(0,j-7):j].mean(axis=0)/5000,
                             np.full(T,np.sin(2*np.pi*j/7)),np.full(T,np.cos(2*np.pi*j/7))],axis=1)
        idx=range(max(7,d-60),d);X=np.concatenate([feat(j) for j in idx]);z=y[list(idx)].reshape(-1)/5000
        reg=np.eye(X.shape[1])*1.;reg[0,0]=1e-6
        coef=np.linalg.solve(X.T@X+reg,X.T@z);v=feat(d)@coef*5000
        # 用历史范围裁剪外推，禁止用当日实际值设定上下界。
        v=np.clip(v,0,np.maximum(h[-30:].max(axis=0)*1.3,1.))
        return np.maximum(v,0)

def build_forecasts(data):
    # 对365日逐一调用因果预测器；生成所有候选，不用外样本选择模型。
    out={}
    for m in METHODS:
        out[m]={}
        for k,col in [('L',1),('PV',2)]:
            out[m][k]=np.stack([forecast(data[k],d,m,data['a'][:,col]) for d in range(365)])
    return out

def metrics(pred,actual):
    # 返回MAE、RMSE、归一化MAE和避开近零实际值的MAPE；功率误差单位kW。
    e=pred-actual;mask=actual>max(1,float(actual.max()))*.01
    return
dict(MAE=float(np.abs(e).mean()),RMSE=float(np.sqrt((e*e).mean())),nMAE=float(np.abs(e).mean()/max(actual.mean(),1)),MAPE=float(np.mean(np.abs(e[mask])/actual[mask]))*100))

def interpolate_hourly_forecast(values,anchor,method='linear'):
    # 用发布时间已知的左端锚点插值整点预报；返回10分钟分辨率未来24小时功率。
    x=np.arange(25);y=np.r_[anchor,values];q=np.arange(1,145)/6
    v=np.interp(q,x,y) if method=='linear' else PchipInterpolator(x,y)(q)
    return np.maximum(v,0)

def scenarios(pred,residuals,margin=.5):
    # 从最近30个完整日残差按日总量排序选代表曲线；margin乘标准差，保持日内相关结构。
    # 最近历史预测残差的真实整日曲线按全天净能量排序选三条代表；不读取目标日。
    if len(residuals)<3:return pred[None,:]
    r=residuals[-30:];order=np.argsort(r.sum(axis=1));ix=np rint(np.array([.15,.5,.85])*(len(r)-1)).astype(int)
    return pred[None,:]+r[order[ix]]+margin*r.std(axis=0)[None,:]
```

附录 C.4 线性规划求解

源文件：optimization.txt；恢复文件：src/optimization.py。

```
"""线性规划求解。输入输出单位沿用论文；发生约束错误时直接报错。"""
from common import *
from data_io import *
from forecasting import *
from common import _minute

@lru_cache(maxsize=128)
def lp_structure(n,k,eta):
    # 按 G、各情景 C/D/R/W/S、终端绝对偏差、增减购电量布局建立稀疏矩阵；按规模和效率缓存。
    # G[n]，各情景(C,D,R,W,S_next)[5n]，terminal_abs[k]，optional plus/minus[2n]
    nvar=n+k*5*n+k*2*n;A=lil_matrix((2*n*k+n,nvar));b=np.zeros(2*n*k+n)
    for s in range(k):
        off=n+s*5*n
        for t in range(n):
            r=2*n*s+t;A[r,t]=1;A[r,off+t]=-1;A[r,off+n+t]=1;A[r,off+2*n+t]=1;A[r,off+3*n+t]=-1
            r=2*n*s+n+t;A[r,off+4*n+t]=1;A[r,off+t]=-eta;A[r,off+n+t]=1/eta
            if t:A[r,off+4*n+t-1]=-1
        plus=n+k*5*n+k
        for t in range(n):A[2*n*k+t,t]=1;A[2*n*k+t,plus+t]=-1;A[2*n*k+t,plus+n+t]=1
        U=lil_matrix((2*k,nvar))
        for s in range(k):
            z=n+k*5*n+s;end=n+s*5*n+5*n-1
            U[2*s,end]=1;U[2*s,z]=-1;U[2*s+1,end]=-1;U[2*s+1,z]=-1
        return A.tocsr(),U.tocsr()

def
optimize_day(net_scenarios,p,s0,eta=.9,capacity=12000,emergency=5,terminal=.5,old=None,fee=.5,refund=Tr
ue,cyclic=False):
    # 共同计划连接各情景；old 为空表示日前，否则对上一有效计划增减。返回计划与首情景状态；该状态不是实际回放。
    N=np.atleast_2d(net_scenarios);k,n=N.shape;p=np.asarray(p);A,U=lp_structure(n,k,eta)
    nv=A.shape[1];c=np.zeros(nv);b=np.zeros(A.shape[0]);bu=np.tile([capacity*.5,-capacity*.5],k)
    weights=np.ones(k)/k;c[:n]=p if old is None else 0
    bounds=[(0,None)]*nv;eps=1e-7
    for s in range(k):
        off=n+s*5*n;b[2*n*s:2*n*s+n]=N[s];b[2*n*s+n]=s0
        c[off:off+2*n]=eps/k;c[off+2*n:off+3*n]=emergency*p*weights[s]
        c[n+k*5*n+s]=terminal*p.mean()*weights[s]
        for t in range(n):
            bounds[off+t]=(0,5000*DT);bounds[off+n+t]=(0,5000*DT)
            bounds[off+4*n+t]=(capacity*.1,capacity*.9)
            if cyclic:bounds[off+2*n+t]=(0,0)
            if cyclic:bounds[off+5*n-1]=(s0,s0)
        plus=n+k*5*n+k
    if old is None:
        # 删除交易关系行；辅助增减量固定零。
        Ause=A[:2*n*k];buse=b[:2*n*k]
        for t in range(2*n):bounds[plus+t]=(0,0)
    else:
        b[2*n*k:]=old;Ause=A;buse=b
        c[plus:plus+n]=(1+fee)*p;c[plus+n:]=(-1+fee if refund else fee)*p

sol=linprog(c,A_ub=U,b_ub=bu,A_eq=Ause,b_eq=buse,bounds=bounds,method='highs',options={'dual_feasibilit
y_tolerance':1e-8,'primal_feasibility_tolerance':1e-8})
if not sol.success:raise RuntimeError(sol.message)
off=n;C=sol.x[off:off+n];D=sol.x[off+n:off+2*n]
assert np.max(np.minimum(C,D))<1e-5,'同时充放电'
return dict(G=np.maximum(sol.x[:n],0),C=C,D=D,S=np.r_[s0,sol.x[off+4*n:off+5*n]],objective=sol.fun)
```

附录 C.5 运行与交易结算

源文件：dispatch.txt；恢复文件：src/dispatch.py。

```
"""运行与交易结算。输入输出单位沿用论文；发生约束错误时直接报错。"""
from common import *
from data_io import *
from forecasting import *
from optimization import *
from common import _minute

def recourse(g,net,s,eta=.9,capacity=12000):
    # 只使用当前净负荷及库存。先充/放电再补购；输出 C,D,R,W 和下一库存，均以 kwh 计。
    """当前时段测量到达后进行局部能量平衡；绝不读取下一时段。"""
    surplus=g-net
    if surplus>=0:
        C=min(surplus,5000*DT,max(0,(capacity*.9-s)/eta));D=R=0.;W=surplus-C
    else:
        D=min(-surplus,5000*DT,max(0,(s-capacity*.1)*eta));C=W=0.;R=-surplus-D
    return C,D,R,W,s+eta*C-D/eta

def merge_emergency_intervals(r,tol=1e-8):
    # 合并相邻紧急购电时段，核验合并前后电量守恒。
    out=[];start=None
    for t in range(len(r)+1):
        active=t<len(r) and r[t]>tol
        if active and start is None:start=t
        if not active and start is not
    None:out.append((interval(start,t),float(r[start:t].sum())));start=None
    assert abs(sum(x[1] for x in out)-r.sum())<1e-6
    return out

def
run_policy(data,f1,fp,margin=.5,terminal=.5,eta=.9,capacity=12000,emergency=5,updates=(),q3=False,price
_actual=False,price_forecast=False,fee=.5,refund=True,interp='linear',days=None,s0=None,tag='',save=True
e):
    # 逐日制定计划并在指定发布时间更新未执行部分；每个 10 分钟时段实测执行，跨日不重置 SOC。save=False 禁止导出。
    if days is None:days=range(365)
    days=list(days);s=capacity*.5 if s0 is None else s0;out={k:[] for k in
['G0','G','C','D','R','W','S','plus','minus','PVhat','Lhat','Pdecision']};daily=[];versions=[]
    netactual=(data['L']-data['PV'])*DT
    resid=netactual-(f1-fp)*DT
    for d in days:
        # 正式评价从 2 月开始。各问题 3 策略在 1 月统一按问题 2 预运行，确保 2 月初状态可比。
        q3_active=q3 and not (days[0]==0 and d<31)
        p=data['P'][d] if price_actual else data['a'][:,0]
        pp=forecast(data['P'],d,'ma7',data['a'][:,0]) if price_forecast else p
        pv0=fp[d].copy()
        if q3_active:pv0=interpolate_hourly_forecast(data['F'][d,0],data['PV'][d-1,-1] if d else
0,interp)
        pred=(f1[d]-pv0)*DT
        # 问题 3 用同一发布时刻过去日期的预报残差，避免把日前误差套在临近预报。
        if q3_active:
            hist=[]
            for j in range(max(0,d-30),d):
                pj=interpolate_hourly_forecast(data['F'][j,0],data['PV'][j-1,-1] if j else 0,interp)
                hist.append(netactual[j]-(f1[j]-pj)*DT)
            rs=np.asarray(hist).reshape(-1,T)
        else:rs=resid[:d]
        m=.5 if d<31 and days[0]==0 else margin;term=.5 if d<31 and days[0]==0 else terminal
```

```

N=scenarios(pred,rs,m)
plan=optimize_day(N,pp,s,eta,capacity,emergency,term);g0=plan['G'].copy();g=g0.copy()

C=np.zeros(T);D=C.copy();R=C.copy();W=C.copy();plus=C.copy();minus=C.copy();S=np.zeros(T+1);S[0]=s;ph=p
v0.copy();lh=f1[d].copy()
vv=[g0.copy()]
for t in range(T):
    if q3_active and t in [int(h*6) for h in updates]:
        hour=t//6
        pvnew=interpolate_hourly_forecast(data['F'][d,hour//6],data['PV'][d,t-1],interp)[:T-t]
        rnow=(data['L'][d,max(0,t-6):t]-f1[d,max(0,t-6):t]).mean()
        lnew=np.maximum(0,f1[d,t:]+rnow*np.exp(-np.arange(T-t)/36))
        prednew=(lnew-pvnew)*DT
        rh=[]
        for j in range(max(0,d-30),d):
            hp=interpolate_hourly_forecast(data['F'][j,hour//6],data['PV'][j,t-1],interp)[:T-t]
            lr=(data['L'][j,max(0,t-6):t]-f1[j,max(0,t-6):t]).mean()
            hl=np.maximum(0,f1[j,t:]+lr*np.exp(-np.arange(T-t)/36))
            rh.append(netactual[j,t:]-hl-hp)*DT
        N=scenarios(prednew,np.asarray(rh).reshape(-1,T-t),m)
        pl=optimize_day(N,pp[t:],s,eta,capacity,emergency,term,old=g[t:],fee=fee,refund=refund)
        delta=pl['G']-g[t:];plus[t:]+=np.maximum(delta,0);minus[t:]+=np.maximum(-
delta,0);g[t:]=pl['G'];ph[t:]=pvnew;lh[t:]=lnew
        vv.append(g.copy())
        C[t],D[t],R[t],W[t],s=recurse(g[t],netactual[d,t],s,eta,capacity);S[t+1]=s
        base=float(p@g0);inc=float((1+fee)*p@plus);cancel=float(fee*p@minus);ref=float(p@minus) if
refund else 0.;em=float(emergency*p@R)
        total=base+inc+cancel-ref+em

daily.append(dict(day=d,date=str(DATES[d].date()),G0=g0.sum(),G=g.sum(),R=R.sum(),W=W.sum(),base=base,i
ncrease=inc,cancel=cancel,refund=ref,adjustment=inc+cancel-
ref,emergency=em,total=total,S0=S[0],Send=S[-
1],C=C.sum(),D=D.sum(),absolute_adjustment=(plus+minus).sum(),surcharge=fee*float(p@(plus+minus)))
    for key,val in
[('G0',g0),('G',g),('C',C),('D',D),('R',R),('W',W),('S',S),('plus',plus),('minus',minus),('PVhat',ph),
('Lhat',lh),('Pdecision',pp)]:out[key].append(val)
    versions.append(vv)
    if tag and (d%60==0 or d==days[-
1]):print(tag,str(DATES[d].date()),'cost',round(total,2),flush=True)
    out={k:np.asarray(v) for k,v in
out.items()};out['days']=np.array(days);out['eta']=np.array(eta);out['capacity']=np.array(capacity);out
['multiplier']=np.array(emergency);out['fee']=np.array(fee);out['refund_flag']=np.array(refund)
    df=pd.DataFrame(daily)
    if save and tag:

np.savez_compressed(ROOT/'results'/f'{tag}.npz',*out);df.to_csv(ROOT/'results'/f'{tag}_daily.csv',inde
x=False,encoding='utf-8-sig')
    dump({str(d):[v.tolist() for v in vv] for d,vv in zip(days,versions) if d in
[78,171,265,354]},ROOT/'results'/f'{tag}_versions.json')
    return out,df

```

附录 C.6 物理与费用验证

源文件：validation.txt；恢复文件：src/validation.py。

```
"""物理与费用验证。输入输出单位沿用论文；发生约束错误时直接报错。"""
from common import *
from data_io import *
from forecasting import *
from optimization import *
from dispatch import *
from common import _minute

def validate_solution(out,data,df,price_actual=False):
    # 从轨迹和实际输入独立重算 11 类等式及边界；超过 1e-6 立即失败，不只依赖优化器成功标志。
    ids=out['days'];p=data['P'][ids] if price_actual else np.broadcast_to(data['a'][:,0],(len(ids),T))
    G,C,D,R,W,S=[out[k] for k in
    ['G','C','D','R','W','S']];eta=float(out['eta']);cap=float(out['capacity']);fee=float(out['fee'])
    bal=G+data['PV'][ids]*DT+D+R-data['L'][ids]*DT-C-W
    expected=(p*out['G0']).sum(axis=1)+(1+fee)*(p*out['plus']).sum(axis=1)+(fee-
    int(out['refund_flag'])*(p*out['minus']).sum(axis=1)+float(out['multiplier'])*(p*R).sum(axis=1)
    checks={'能量守恒':np.max(np.abs(bal)), 'SOC 动态':np.max(np.abs(S[:,1:]-S[:, :-1]-eta*C+D/eta)),
    'SOC 边界':max(0, cap*.1-S.min(), S.max()-cap*.9), '充放电上限':max(0, C.max()-5000*DT, D.max()-
    5000*DT),
    '非负':max(0, -min(v.min() for v in [G,C,D,R,W])), '同时充放电':np.minimum(C,D).max(),
    '跨日连续':np.max(np.abs(S[1:,0]-S[:, -1, -1])) if np.all(np.diff(ids)==1) and len(ids)>1 else
    0.,
    '费用独立核算':np.max(np.abs(expected-df.total.to_numpy())),
    '有效计划交易恒等式':np.max(np.abs(G-out['G0']-out['plus']+out['minus'])),
    '紧急合并守恒':max(abs(sum(v for _,v in merge_emergency_intervals(r))-r.sum()) for r in R),
    '有限数值':float(not all(np.isfinite(x).all() for x in [G,C,D,R,W,S]))}
    result=[dict(检查=k, 误差=float(v), 状态='PASS' if v<1e-6 else 'FAIL') for k,v in checks.items()]
    assert all(x['状态']=='PASS' for x in result),result
    return result
```

附录 C.7 模块统一接口

源文件：model.txt；恢复文件：src/model.py。

```
"""兼容入口：向原主程序导出分功能模块。"""  
from common import *  
from data_io import *  
from forecasting import *  
from optimization import *  
from dispatch import *  
from validation import *
```

附录 C.8 四问计算 参数选型 灵敏度实验

源文件：run.txt；恢复文件：src/run.py。

本模块负责阶段调度；all 从输入重建计算管线，annual 优先读取已有轨迹，annual_force 强制重算。为复核结果，应保留运行时依赖版本和输入校验值。

```
from model import *
import argparse, time

def main(stage):
    data=load_data()
    if stage in ['all','q1']:
        a=data['a'];r=optimize_day((a[:,1]-a[:,2])*DT,a[:,0],6000,cyclic=True,terminal=0)
        G,C,D,S=[r[x] for x in ['G','C','D','S']];W=G+a[:,2]*DT+D-a[:,1]*DT-C
        assert W.min()>-1e-6 and abs(S[-1]-6000)<1e-6
        np.savez_compressed(ROOT/'results'/'q1.npz',G=G,C=C,D=D,S=S,W=W)
        summary=dict(total_energy=G.sum(),cost=a[:,0]@G,initial=S[0],final=S[-1],max_balance_error=float(np.max(np.abs(G+a[:,2]*DT+D-a[:,1]*DT-C-W))),no_battery_cost=float(a[:,0]@np.maximum((a[:,1]-a[:,2])*DT,0)),six_slots={interval(t):G[t] for t in [60,72,84,96,108,120]})
        # 左端样本、日内周期闭合的另一解释，仅作参考敏感性，不用于主模板。
        alt=np.roll(a,1,axis=0);rr=optimize_day((alt[:,1]-alt[:,2])*DT,alt[:,0],6000,cyclic=True,terminal=0)
        summary['left_endpoint_alternative_cost']=alt[:,0]@rr['G']
        dump(summary,ROOT/'results'/'q1_summary.json');print('Q1',summary,flush=True)
        pd.DataFrame([dict(区间=interval(t,t+24),充电量=C[t:t+24].sum(),放电量=D[t:t+24].sum()) for t in range(0,144,24)]).to_csv(ROOT/'results'/'表 2_问题 1.csv',index=False,encoding='utf-8-sig')
        if stage=='q1':return
    if stage in ['all','forecast']:
        forecasts=build_forecasts(data)
        np.savez_compressed(ROOT/'results'/'forecasts.npz',{m+'_'+k:v for m,dic in forecasts.items() for k,v in dic.items()})
        rows=[]
        for m in METHODS:
            for k in ['L','PV']:
                for period,sl in [(('1 月滚动验证',slice(16,31)),('2 月至 12 月外样本',slice(31,None))):
                    rows.append(dict(method=m,variable=k,period=period,**metrics(forecasts[m][k][sl],data[k][sl])))
        pd.DataFrame(rows).to_csv(ROOT/'results'/'预测精度.csv',index=False,encoding='utf-8-sig')
        # 固定相同初始 SOC，连续 15 日经济回溯；不重置每一天的 SOC。
        tune=[]
        for m in METHODS:
            for margin in [0.,.5,1.]:
                for terminal in [0.,.5,1.]:
                    _,df=run_policy(data,forecasts[m]['L'],forecasts[m]['PV'],margin=margin,terminal=terminal,days=range(16,31),save=False)
                    tune.append(dict(method=m,margin=margin,terminal=terminal,cost=df.total.sum(),R=df.R.sum(),Send=df.Send.iloc[-1]))
        print('tuned',m,flush=True)
        td=pd.DataFrame(tune);td['inventory_adjusted_cost']=td.cost-.5*data['a'][:,0].mean()*(td.Send-6000)
        td=td.sort_values('inventory_adjusted_cost');td.to_csv(ROOT/'results'/'1 月经济选型.csv',index=False,encoding='utf-8-sig')
        selected=td.iloc[0].to_dict();dump(selected,ROOT/'results'/'selected.json');print('SELECTED',selected,flush=True)
        if stage=='forecast':return
    f=np.load(ROOT/'results'/'forecasts.npz');selected=json.loads((ROOT/'results'/'selected.json').read_text())
```

```

t(encoding='utf-8'))
m=selected['method'];fl=f[m+'_L'].copy();fp=f[m+'_PV'].copy()
# 所有正式策略 1 月统一使用事先固定 EWMA; 不会用 1 月末选型反写 1 月的 SOC。
fl[:31]=f['ewma_L'][:31];fp[:31]=f['ewma_PV'][:31]
common=dict(margin=selected['margin'],terminal=selected['terminal'])
if stage in ['all','annual','annual_force']:
    validations=[];summaries=[]

experiments=[('q2',False,False,()),('q3_A',True,False,()),('q3_B',True,False,(6,)),('q3_C',True,False,(6,12)),('q3_D',True,False,(6,12,18)),('q42',False,True,()),('q43_A',True,True,()),('q43_B',True,True,(6,)),('q43_C',True,True,(6,12)),('q43_D',True,True,(6,12,18)),('q42_causal_price',False,True,()),('q43_causal_price',True,True,(6,12,18))]
    for tag,q3,pa,updates in experiments:
        if stage!='annual_force' and (ROOT/'results'/f'{tag}.npz').exists():

out=dict(np.load(ROOT/'results'/f'{tag}.npz'));df=pd.read_csv(ROOT/'results'/f'{tag}_daily.csv')
    else:out,df=run_policy(data,fl,fp,**common,q3=q3,price_actual=pa,price_forecast='causal' in
tag,updates=updates,tag=tag)
    validations.extend([dict(model=tag,**v) for v in validate_solution(out,data,df,pa)])

agg=df[df.day>=31].select_dtypes('number').sum().to_dict();agg.pop('day');agg.pop('S0');agg.pop('Send')
    summaries.append(dict(model=tag,**agg,initial_Feb=out['S'][31,0],final_Dec=out['S'][-1,-1]))

    pd.DataFrame(validations).to_csv(ROOT/'audit'/'物理与费用验证.csv',index=False,encoding='utf-8-sig')
    pd.DataFrame(summaries).to_csv(ROOT/'results'/'年度汇总.csv',index=False,encoding='utf-8-sig')
    if stage in ['all','sensitivity']:
        # 24 个连续代表日: 四季各 6 日, 初值取主策略同日已实现 SOC; 同一组比较统一初值。
        starts=[76,169,263,352];rows=[]

cases=[('baseline',{},{},{'eta085',{ 'eta':.85}},{ 'eta095',{ 'eta':.95}},{ 'roundtrip090',{ 'eta':np.sqrt(.9)}},{ 'capacity9600',{ 'capacity':9600}},{ 'capacity14400',{ 'capacity':14400}},{ 'capacity18000',{ 'capacity':18000}},{ 'emergency3',{ 'emergency':3}},{ 'emergency7',{ 'emergency':7}},{ 'margin0',{ 'margin':0}},{ 'margin1',{ 'margin':1}},{ 'terminal0',{ 'terminal':0}},{ 'terminal1',{ 'terminal':1}},{ 'fee025',{ 'fee':.25}},{ 'fee075',{ 'fee':.75}},{ 'no_refund',{ 'refund':False}},{ 'pchip',{ 'interp':'pchip'}},{ 'error150',{}}])
    for tag,q3 in [ ('q2',False),('q3_D',True)]:
        base=np.load(ROOT/'results'/f'{tag}.npz')
        for name,kw in cases:
            if not q3 and name in ['fee025','fee075','no_refund','pchip']:continue
            opt=common|kw;cost=em=ener=spill=0.;value=0.
            for start in starts:
                load=fl.copy();pv=fp.copy()
                # 压力试验只改变当时已可用预测的振幅, 不读取当日实际误差。
                if name=='error150':load=load*1.10;pv=pv*.90

cap=opt.get('capacity',12000);sinit=float(np.clip(base['S'][start,0],cap*.1,cap*.9))
                o,d=run_policy(data,load,pv,**opt,q3=q3,updates=(6,12,18) if q3 else
(),days=range(start,start+6),s0=sinit,save=False)

validate_solution(o,data,d);cost+=d.total.sum();em+=d.emergency.sum();ener+=d.R.sum();spill+=d.W.sum();
                value+=.5*data['a'][:,0].mean()*(d.Send.iloc[-1]-sinit)
                rows.append(dict(model=tag,case=name,cost=cost,inventory_adjusted_cost=cost-value,emergency=em,R=ener,W=spill))
                print('sensitivity',tag,name,round(cost,2),flush=True)
                pd.DataFrame(rows).to_csv(ROOT/'results'/'敏感性 24 日.csv',index=False,encoding='utf-8-sig')
                print('DONE',stage,flush=True)

if __name__=='__main__':
    p=argparse.ArgumentParser();p.add_argument('--stage',default='all');main(p.parse_args().stage)

```

附录 C.9 独立求解与原始对偶检验

源文件: q1_certificate.txt; 恢复文件: src/q1_certificate.py。

```
"""问题 1 可复现求解。运行 python solve_q1.py, 依赖 numpy scipy openpyxl matplotlib."""
from pathlib import Path
import sys, json, csv, hashlib
HERE = Path(__file__).resolve().parent
if (HERE / 'pydeps').exists():
    sys.path.insert(0, str(HERE / 'pydeps'))
import numpy as np
import scipy
from scipy.optimize import linprog
import openpyxl

ROOT = HERE.parent
OUT = HERE.parent / '计算结果'
OUT.mkdir(exist_ok=True)

def solve(price, load, pv, eta=.9, power=5000., initial=6000., cyclic=True, method='highs-ds'):
    # x = [grid(144), charge(144), discharge(144), spill(144), energy(145)]
    n = len(price); nv = 5*n+1; dt = 1/6
    A = np.zeros((2*n, nv)); b = np.zeros(2*n)
    for t in range(n):
        A[t, [t, n+t, 2*n+t, 3*n+t]] = [1, -1, 1, -1]
        b[t] = (load[t]-pv[t])*dt
        A[n+t, [n+t, 2*n+t, 4*n+t, 4*n+t+1]] = [-eta, 1/eta, -1, 1]
    bounds = ([ (0, None)]*n + [(0, power*dt)]*(2*n)
               + [(0, float(v*dt)) for v in pv] + [(1200, 10800)]*(n+1))
    bounds[4*n] = (initial, initial)
    if cyclic: bounds[-1] = (initial, initial)
    c = np.r_[price, np.zeros(4*n+1)]
    r = linprog(c, A_eq=A, b_eq=b, bounds=bounds, method=method)
    if not r.success: raise RuntimeError(r.message)
    x = r.x; q, ch, dis, spill = np.split(x[:4*n], 4); e = x[4*n:]
    dual = float(b@r.eqlin.marginals)
    for j, (lo, hi) in enumerate(bounds):
        if lo is not None: dual += lo*r.lower.marginals[j]
        if hi is not None: dual += hi*r.upper.marginals[j]
    balance = q + pv*dt + dis - load*dt - ch - spill
    dyn = np.diff(e) - eta*ch + dis/eta
    checks = {'balance_max_abs_kwh': float(np.max(np.abs(balance))),
              'storage_max_abs_kwh': float(np.max(np.abs(dyn))),
              'initial_kwh': float(e[0]), 'terminal_kwh': float(e[-1]),
              'minimum_storage_kwh': float(e.min()), 'maximum_storage_kwh': float(e.max()),
              'maximum_charge_kw': float(ch.max()/dt), 'maximum_discharge_kw': float(dis.max()/dt),
              'simultaneous_charge_discharge_slots': int(np.sum((ch>1e-7)&(dis>1e-7))),
              'dual_objective_yuan': dual, 'primal_dual_gap_yuan': float(abs(r.fun-dual)),
              'global_energy_residual_kwh': float(q.sum()+pv.sum()*dt-load.sum()*dt-spill.sum()-(1-eta)*ch.sum()-(1/eta-1)*dis.sum()-(e[-1]-e[0]))}
    return {'cost': float(r.fun), 'purchase': float(q.sum()), 'charge': float(ch.sum()),
            'discharge': float(dis.sum()), 'spill': float(spill.sum()), 'q': q.tolist(),
            'c': ch.tolist(), 'd': dis.tolist(), 'w': spill.tolist(), 'e': e.tolist(), 'checks': checks}

def main():
    p = ROOT/'input'/'附件 1.xlsx'
    ws = openpyxl.load_workbook(p, data_only=True).active
    rows = list(ws.values)[1:]
    assert len(rows) == 144
    a = np.array([r[1:] for r in rows], dtype=float)
    price, load, pv = a.T
    assert np.isfinite(a).all() and (a>=0).all()
    def minute(v):
```

```

        if hasattr(v, 'hour'): return v.hour*60+v.minute
        return 1440 if '+1' in v else int(v.split(':')[0])*60+int(v.split(':')[1])
    assert [minute(r[0]) for r in rows]==list(range(10,1441,10))
    res=solve(price,load,pv)
    baseline=np.maximum(load-pv,0)/6
    res.update({'load_total':float(load.sum()/6), 'pv_total':float(pv.sum()/6),
               'baseline_cost':float(price@baseline), 'baseline_purchase':float(baseline.sum()),
               'baseline_spill':float(np.maximum(pv-load,0).sum()/6),
               'price_min':float(price.min()), 'price_max':float(price.max()),
               'source_sha256':hashlib.sha256(p.read_bytes()).hexdigest(),
               'scipy_version':scipy.__version__, 'numpy_version':np.__version__,
               'price':price.tolist(), 'load':load.tolist(), 'pv':pv.tolist()})
    res['savings']=res['baseline_cost']-res['cost']
    res['savings_pct']=100*res['savings']/res['baseline_cost']
    res['blocks']=[{'period':f'{i*4}:00-{(i+1)*4}:00',
                    'charge':sum(res['c'][i*24:(i+1)*24]),
                    'discharge':sum(res['d'][i*24:(i+1)*24])} for i in range(6)]
    res['selected']=[{'period':f'{h}:00-{h}:10', 'q':res['q'][h*6]} for h in [10,12,14,16,18,20]]
    sensitivity=[]
    for label,eta,power,initial,cyclic in [
        ('充放电各 85%', .85,5000,6000,True), ('充放电各 90%', .9,5000,6000,True),
        ('往返效率 90%', np.sqrt(.9),5000,6000,True), ('最大功率 4000kw', .9,4000,6000,True),
        ('最大功率 6000kw', .9,6000,6000,True), ('初始电量 1200kWh', .9,5000,1200,True),
        ('初始电量 10800kWh', .9,5000,10800,True), ('不约束日未回归', .9,5000,6000,False)]:
        z=solve(price,load,pv,eta,power,initial,cyclic)
        sensitivity.append({'case':label, 'cost':z['cost'], 'purchase':z['purchase'], 'end':z['e'][-1]})
    shifted=solve(np.roll(price,1),np.roll(load,1),np.roll(pv,1))
    sensitivity.append({'case':'时刻作区间起点并循环补 0 时',
                        'cost':shifted['cost'], 'purchase':shifted['purchase'], 'end':shifted['e'][-1]})
    res['sensitivity']=sensitivity
    alt=solve(price,load,pv,method='highs-ipm')
    res['checks']['independent_algorithm_cost_gap_yuan']=abs(alt['cost']-res['cost'])
    assert res['checks']['balance_max_abs_kwh']<1e-6
    assert res['checks']['storage_max_abs_kwh']<1e-6
    assert res['checks']['primal_dual_gap_yuan']<1e-5
    assert res['checks']['simultaneous_charge_discharge_slots']==0
    assert abs(res['e'][-1]-6000)<1e-6
    (OUT/'summary.json').write_text(json.dumps(res,ensure_ascii=False,indent=2),encoding='utf-8')
    def tm(m): return f'{m//60:02d}:{m%60:02d}'
    with (OUT/'逐时段策略.csv').open('w',encoding='utf-8-sig',newline='') as f:
        w=csv.writer(f);w.writerow(['序号','时间段','电价元每 kWh','负载 kW','光伏 kW','购电 kWh','充电 kWh',
                                     '放电 kWh','弃光 kWh','段初 SOC_kWh','段末 SOC_kWh','购电费元'])
        for t in range(144):w.writerow([t+1,tm(10*t)+'-'+tm(10*(t+1)),price[t],load[t],pv[t],res['q'][t],res['c'][t],res['d'][t],res['w'][t],res['e'][t],res['e'][t+1],price[t]*res['q'][t]])
    import matplotlib
    matplotlib.use('Agg')
    import matplotlib.pyplot as plt
    plt.rcParams.update({'font.sans-serif':['Microsoft YaHei'], 'axes.unicode_minus':False, 'font.size':10})
    t=np.arange(144)/6
    fig,axs=plt.subplots(3,1,figsize=(9,7),sharex=True,constrained_layout=True)
    axs[0].step(t,load,where='post',label='小区负载',color='#283747')
    axs[0].step(t,pv,where='post',label='光伏预测功率',color='#da8b00')
    axs[0].step(t,np.array(res['q'])*6,where='post',label='计划购电功率',color='#2874a6',alpha=.85)
    axs[0].set_ylabel('功率 / kW');axs[0].legend(ncol=3,fontsize=9)
    axs[1].bar(t,np.array(res['c'])*6,width=1/6,color='#2980b9',label='充电')
    axs[1].bar(t,-np.array(res['d'])*6,width=1/6,color='#cb4335',label='放电')
    axs[1].set_ylabel('储能功率 / kW');axs[1].legend(ncol=2)
    axs[2].plot(np.arange(145)/6,res['e'],color='#1e8449',label='储电量')
    axs[2].axhline(1200,color='gray',ls='--');axs[2].axhline(10800,color='gray',ls='--')
    axs[2].set_ylabel('储电量 / kWh');axs[2].set_xlabel('当日时刻 / h')
    ax=axs[2].twinx();ax.step(t,price,where='post',color='#7d3c98',alpha=.55);ax.set_ylabel('电价 / 元每 kWh')
    for a in axs:a.grid(alpha=.2);a.set_xlim(0,24);a.set_xticks(range(0,25,2))
    fig.savefig(OUT/'调度曲线.png',dpi=200);plt.close(fig)

```

```
    print(json.dumps({k:res[k] for k in
['cost','purchase','charge','discharge','spill','baseline_cost','savings_pct','blocks','selected','checks','sensitivity']}),ensure_ascii=False,indent=2))

if __name__=='__main__':
    sys.stdout.reconfigure(encoding='utf-8')
    main()
```

附录 C.10 压力 误差 重采样及风险模型扩展

源文件：extended_checks.txt；恢复文件：src/extended_checks.py。

本模块将压力试验、误差统计、配对区块重采样和风险权重实验分为独立函数。

权重选择使用 1 月，四季 24 日用于后续验证，固定随机种子保证重采样复现。

```
"""四方面检验与风险惩罚扩展。固定种子；原结果只读；不改写提交表。"""
from pathlib import Path
import sys, inspect, json
import numpy as np
import pandas as pd
from scipy.sparse import hstack, vstack, csr_matrix
HERE=Path(__file__).resolve().parent
import dispatch as m
from validation import validate_solution
m.validate_solution=validate_solution
SRC=HERE
R=HERE.parent/'results'
OUT=HERE.parent/'检验结果';OUT.mkdir(exist_ok=True)

ORIGINAL_OPTIMIZER=m.optimize_day
def risk_solver(weight):
    """增加最坏情景紧急费用变量 z：目标为原目标加 weight*z，仍为 LP。"""
    source=inspect.getsource(ORIGINAL_OPTIMIZER)
    insertion=''
    if risk_weight > 0:
        # 最后一列是最坏情景损失上界；情景共同日前计划保持不变。
        c=np.r_[c,risk_weight];bounds=bounds+[(0,None)]
        Ause=hstack([Ause,csr_matrix((Ause.shape[0],1))],format='csr')
        U=hstack([U,csr_matrix((U.shape[0],1))],format='csr')
        tail=np.zeros((k,nv+1))
        for ss in range(k):
            off0=n+ss*5*n
            tail[ss,off0+2*n:off0+3*n]=emergency*p
            tail[ss,-1]=-1
        U=vstack([U,csr_matrix(tail)],format='csr');bu=np.r_[bu,np.zeros(k)]
    ...
    source=source.replace(' sol=linprog(',insertion+' sol=linprog(')
    ns=dict(vars(m),risk_weight=weight,hstack=hstack,vstack=vstack,csr_matrix=csr_matrix)
    exec(compile(source,'risk_optimize_day','exec'),ns)
    return ns['optimize_day']

def evaluate_risk(data,f1,fp):
    """权重只在 1 月选择；随后在预先指定的四季 24 日验证，初态一致。"""
    original=m.optimize_day;records=[]
    try:
        for w in [0,.1,.3,1.]:
            m.optimize_day=risk_solver(w) if w else original
    _,df=m.run_policy(data,f1,fp,days=range(16,31),s0=6000,q3=True,updates=(6,12,18),margin=0,save=False)
    cost=df.total.sum()-.5*data['a'][:,0].mean()*(df.Send.iloc[-1]-6000)
    records.append(dict(stage='1 月验证',weight=w,cost=cost,emergency=df.emergency.sum()))
    print('risk validation',records[-1],flush=True)
    selected=min(records,key=lambda x:x['cost'])['weight']
    # 同时报告各候选的外样本表现，不用外样本重新选择权重。
    for w in [0,.1,.3,1.]:
        m.optimize_day=risk_solver(w) if w else original
        cost=em=0.;checks=[]
        for start in [76,169,263,352]:
            initial=float(np.load(R/'q3_D.npz')['S'][start,0])
```

```

out,df=m.run_policy(data,f1,fp,days=range(start,start+6),s0=initial,q3=True,updates=(6,12,18),margin=0,
save=False)

        checks+=m.validate_solution(out,data,df)
        cost+=df.total.sum()-0.5*data['a'][:,0].mean()*(df.Send.iloc[-1]-initial)
        em+=df.emergency.sum()
        records.append(dict(stage='四季 24 日',weight=w,cost=cost,emergency=em))
        print('risk test',records[-1],flush=True)
        pd.DataFrame(records).to_csv(OUT/'风险模型对比.csv',index=False,encoding='utf-8-sig')
        return dict(selected_weight=selected,records=records)
    finally:m.optimize_day=original

def fixed_plan_stress(data):
    """冻结已制定的常规计划，在实际净负荷受扰动时连续执行实时电池控制。"""
    z=np.load(R/'q3_D.npz');g=z['G'][:31];p=data['a'][:,0]
    records=[]
    for label,lf,pvf in [('原始负荷',1,1),('负载增加 5%',1.05,1),('光伏减少 10%',1,.9),('联合不利扰动',1.05,.9)]:
        s=float(z['S'][:31,0]);em=0.;energy=0.;smin=smax=s;res=0.
        for j,d in enumerate(range(31,365)):
            for t in range(144):
                net=(lf*data['L'][d,t]-pvf*data['PV'][d,t])/6
                c,dd,r,w,s=m.recourse(g[j,t],net,s)
                em+=5*p[t]*r;energy+=r;smin=min(smin,s);smax=max(smax,s)
                res=max(res,abs(g[j,t]+dd+r-net-c-w))

    records.append(dict(case=label,emergency_kwh=energy,emergency_cost=em,min_soc=smin,max_soc=smax,balance_error=res,end_soc=s))
    pd.DataFrame(records).to_csv(OUT/'固定计划压力检验.csv',index=False,encoding='utf-8-sig')
    return records

def errors_and_comparison(data,f1,fp):
    """误差按整日聚合；比较采用配对的 7 日移动区块重采样，保留短期相关性。"""
    e=((f1-fp)-(data['L']-data['PV']))[:31:]

    metrics=dict(MAE=float(np.abs(e).mean()),RMSE=float(np.sqrt((e**2).mean()))),bias=float(e.mean()),p95_abs=float(np.quantile(np.abs(e),.95)))
    np.savez_compressed(OUT/'误差数组.npz',net_error=e)
    a=pd.read_csv(R/'q3_A_daily.csv').iloc[:31];d=pd.read_csv(R/'q3_D_daily.csv').iloc[:31:]
    # 同一终端库存价值逐日校正，避免跨日库存差异混入收益。
    price=.5*data['a'][:,0].mean()
    gain=(a.total-price*(a.Send-a.S0))-(d.total-price*(d.Send-d.S0))
    gain=gain.to_numpy();rng=np.random.default_rng(20260912);means=[]
    for _ in range(2000):
        ix=np.concatenate([np.arange(s,s+7) for s in rng.integers(0,len(gain)-6,size=48)])[:len(gain)]
        means.append(gain[ix].mean())
    ci=np.quantile(means,[.025,.975])

    comp=dict(mean_gain=float(gain.mean()),ci_low=float(ci[0]),ci_high=float(ci[1]),positive_days=int((gain>0).sum()),days=len(gain),replicates=2000,block_days=7)
    pd.DataFrame(dict(date=d.date,gain=gain)).to_csv(OUT/'日费用配对差.csv',index=False,encoding='utf-8-sig')
    np.savez_compressed(OUT/'区块重采样.npz',means=means)
    return metrics,comp

def main():

    data=dict(np.load(R/'data.npz'));fc=np.load(R/'forecasts.npz');f1=fc['ridge_L'].copy();fp=fc['ridge_PV'].copy()
    # 沿用既有 1 月预运行信息，不改变历史状态。
    f1[:31]=fc['ewma_L'][:31];fp[:31]=fc['ewma_PV'][:31]
    metrics,comparison=errors_and_comparison(data,f1,fp)

    result=dict(errors=metrics,comparison=comparison,stress=fixed_plan_stress(data),risk=evaluate_risk(data,f1,fp))
    (OUT/'新增检验汇总.json').write_text(json.dumps(result,ensure_ascii=False,indent=2),encoding='utf-8')

```

```
    print(json.dumps(result,ensure_ascii=False),flush=True)
if __name__=='__main__':main()
```

附录 C.11 未来信息污染检验

源文件：causality.txt；恢复文件：src/causality.py。

```
from model import *
def causality():

data=load_data();f=np.load(ROOT/'results'/'forecasts.npz');sel=json.loads((ROOT/'results'/'selected.json').read_text(encoding='utf-8'));checks=[]
    for m in METHODS:
        for key,col in [('L',1),('PV',2)]:
            for d in [31,78,171,265,354]:
                altered=data[key].copy();altered[d]=1e9
                err=np.max(np.abs(forecast(data[key],d,m,data['a'][:,col])-
forecast(altered,d,m,data['a'][:,col])))
                checks.append(dict(检查=f'{m}/{key}/d={d}未来污染',误差=err,状态='PASS' if err==0 else
'FAIL'))

fl=f[sel['method']+'_L'].copy();fp=f[sel['method']+'_PV'].copy();fl[:31]=f['ewma_L'][:31];fp[:31]=f['ewma_PV'][:31]

opts=dict(days=[78],q3=True,updates=(6,12,18),margin=sel['margin'],terminal=sel['terminal'],save=False)
    o,_=run_policy(data,fl,fp,**opts)
    alt={k:v.copy() for k,v in
data.items()};alt['L'][78,72:]*=1.7;alt['PV'][78,72:]*=.3;alt['F'][78,2:]*=1.5
    b,_=run_policy(alt,fl,fp,**opts)
    err=max(np.max(np.abs(o['G0']-b['G0'])),np.max(np.abs(o['G'][:,72]-
b['G'][:,72])),np.max(np.abs(o['S'][:,73]-b['S'][:,73])))
    checks.append(dict(检查='12 时后实际值和预报污染不改变此前计划与状态',误差=err,状态='PASS' if err<1e-6
else 'FAIL'))
    assert all(x['状态']=='PASS' for x in checks)
    pd.DataFrame(checks).to_csv(ROOT/'audit'/'因果性污染测试.csv',index=False,encoding='utf-8-sig')
    dump(dict(selection_cutoff='2025-01-31 24:00',evaluation_start='2025-02-01
00:00',history_rule='strictly before day d, endpoint at day d 00:00 permitted',Jan_policy='EWMA,
margin .5, terminal .5; not retrospectively changed',price_assumption='Q4 main: known day curve; causal
extensions separate',template_last_column='user-approved erratum: current day 00:00-
00:10'),ROOT/'audit'/'信息集.json')
    print('causality PASS',len(checks))
if __name__=='__main__':causality()
```